SignalForge

Functionality walkthrough

Every screen of the incident command centre, photographed from the running product, with what it does and whether it is built.

BUILD `6e6f2f1 · v0.2.0`	CAPTURED 2 September 2026
SCREENS PHOTOGRAPHED 37, live, 0 page errors	REQUIREMENTS 52 of 52, all marked built
TEST SUITE 1,258 passed, 5 skipped, 0 failed	SOURCE 91,515 lines across 507 files

READ THIS FIRST

What this document is, and how it was made

> A build check, not a brochure. Every picture in it was taken from a running instance of SignalForge on 2 September 2026, by signing in as a real user and walking to each route the way a person walks to it. Nothing here is a mockup and nothing was staged. Where a screen had nothing to show, it was photographed showing nothing and labelled as such.

The method

Instance	`cockpit :3000 · API :8000 · reference harness :8080`
Signed in as	`owner@drill.test (Org Owner), sme-db@demo.test (SME, mobile)`
Organisation	`drill-co-2 / payments`
Incident	`INC-000001 - P0, active, declared by a person`
Captures	**37 screens at 1440×900 and 390×844, full page**
Page errors	**0 · no capture landed back on /login**
Test suite	**re-run for this document: 1,258 passed, 5 skipped, 0 failed, 20m09s**

Tall screens are cut into page-shaped bands with a small overlap and laid out in order, because a picture of a screen too small to read has not shown the screen. Nothing is cropped away: every band of every capture is here.

What the badges mean

BUILT · VERIFIED LIVE	The screen answered, with data from the running application in it.
BUILT · EMPTY IN THIS CAPTURE	The screen answered and works; the demonstration world had nothing to put in it at that moment. The reason is given each time.
NOT BUILT	Absent. Section H lists every one of these with its reason.

Two things the product does that shape every screen

The platform proposes; people decide. It never declares an incident, never executes a high-risk action on its own, never sends a communication and never closes an incident. Each of those four is a person pressing something, and the refusal is in the domain rather than in the user interface — so it holds whatever the caller is.

The AI is an accelerator, never a dependency. With no model configured every screen still works and says `degraded`. Detection, hypothesis ranking, policy, the Tool Gateway, validation and replay never used a model in the first place. The captures in this document were taken with the model path degraded, which is why the badge appears — and why every one of those surfaces is nevertheless full.

SECTION 0

The build, at a glance

> Fourteen capability areas, eight sprints, one final commit. Every row was exercised in the captures that follow or by the test suite named beside it.

37	52	107	1258
SCREENS CAPTURED LIVE	REQUIREMENTS, ALL BUILT	API PATHS	TESTS PASSING

AREA	WHAT IS BUILT	STATE	REQUIREMENTS
Identity and tenancy	Organisations, invitations, sessions, seven roles, per-application role bindings, and tenant isolation enforced at the repository layer rather than in each route.	BUILT	`REQ-010, 012, 013`
Secrets and ingest	Credentials encrypted at rest and never read back; push ingest authenticated by HMAC with idempotent receipts; keys rotatable.	BUILT	`REQ-015, 017, 032`
The audit chain	Every consequential act hash-chained. The link is chosen inside the insert by a scalar subquery, so concurrent writers cannot fork it. Verifiable, sealable, replayable.	BUILT	`REQ-016, 023, 072`
Event backbone	Transactional outbox and dispatcher, ordered at-least-once delivery with idempotent handlers, retries, dead letters, catch-up, and a virtual clock for drills.	BUILT	`REQ-020 to 024`
Integrations	Thirteen adapter kinds behind one SPI; a registry with manifests, capabilities and credential scopes; pull, push and simulated adapters; per-connector health measured in the right direction.	BUILT	`REQ-002, 030 to 033, 035`
Onboarding	A seven-step wizard: basics, connect, map, topology, SLOs and policy, baseline, test. Re-entrant, partial-success aware, and proven against a reference application that shares no code with the platform.	BUILT	`REQ-036, 038`
Detection	Rollups, baselines that hold while a signal breaches, anomaly only on the third sustained evaluation, SLO fast and slow burn, corroboration across independent families, weighted health score, alert-storm clustering.	BUILT	`REQ-040 to 042, 046`
Incident lifecycle	Six states in two halves. Declaring, changing severity and closing are reserved for people. ITSM ticket sync and change-window tracking from deploy hooks.	BUILT	`REQ-043, 044`
War room	Membership derived from blast radius, paging with acknowledgement countdowns, automatic escalation through the rotation, a join brief for late arrivals, and bridge commitments captured as owned tasks.	BUILT	`REQ-045`
Incident brain	A versioned graph with provenance on every node; confidence computed deterministically and shown as arithmetic; evidence gathered broad then narrow under a budget, with redaction; a grounded copilot that refuses rather than guesses.	BUILT	`REQ-050, 052 to 055`
Governance and execution	A hardcoded floor no policy can lower, per-application policy above it, one Tool Gateway door, UNCERTAIN outcomes that refuse retry until reconciled, next-best-action scored as arithmetic, governed communications, four-layer validation with a customer gate on closure.	BUILT	`REQ-060 to 065`
Drills, PIR and release	A drill engine on a virtual clock with chaos locked to drill scope by contract; the Recovery Collision drill as pure data; a sixteen-section review generated from the record that refuses to run on a broken chain.	BUILT	`REQ-070 to 073`

The cockpit	Nineteen routes, one typed client, one WebSocket store, tokens extracted from the design export, and six role views of the same room.	BUILT	REQ-007, 045
The model path	One provider adapter: OpenAI-compatible calls, structured outputs, retries, SQLite cache by input hash, a per-incident token budget, and a deterministic fallback that emits system.llm.degraded.	BUILT, DEGRADED HERE	REQ-051

Requirement ids are the frozen catalogue in `specs/requirements.yaml`. All 52 carry `@pytest.mark.req` markers in the suite — 908 marker occurrences across 170 test files.

SECTION A

Getting in

> Identity is multi-tenant and the tenant is part of the credential. A session belongs to one organisation, and every repository read is scoped to it.

S1 Landing

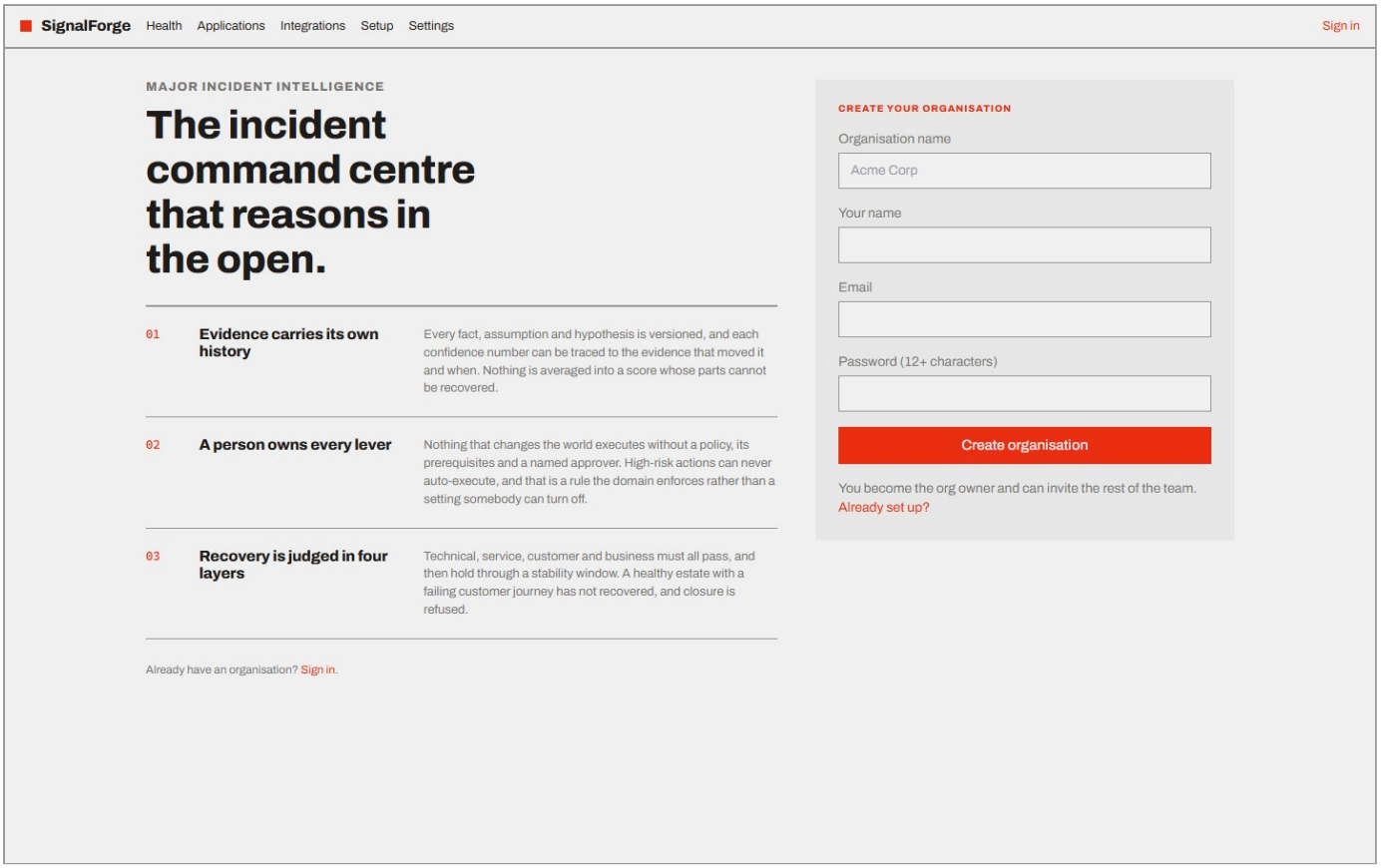
/

BUILT · VERIFIED LIVE

The signed-out front door. It states the three rules the rest of the product is built on before it asks for anything: the platform proposes and a person decides, the AI is an accelerator and never a dependency, and the record is hash-chained.

IN THIS CAPTURE

Signed out, with sign-up and sign-in as the only two things to do.



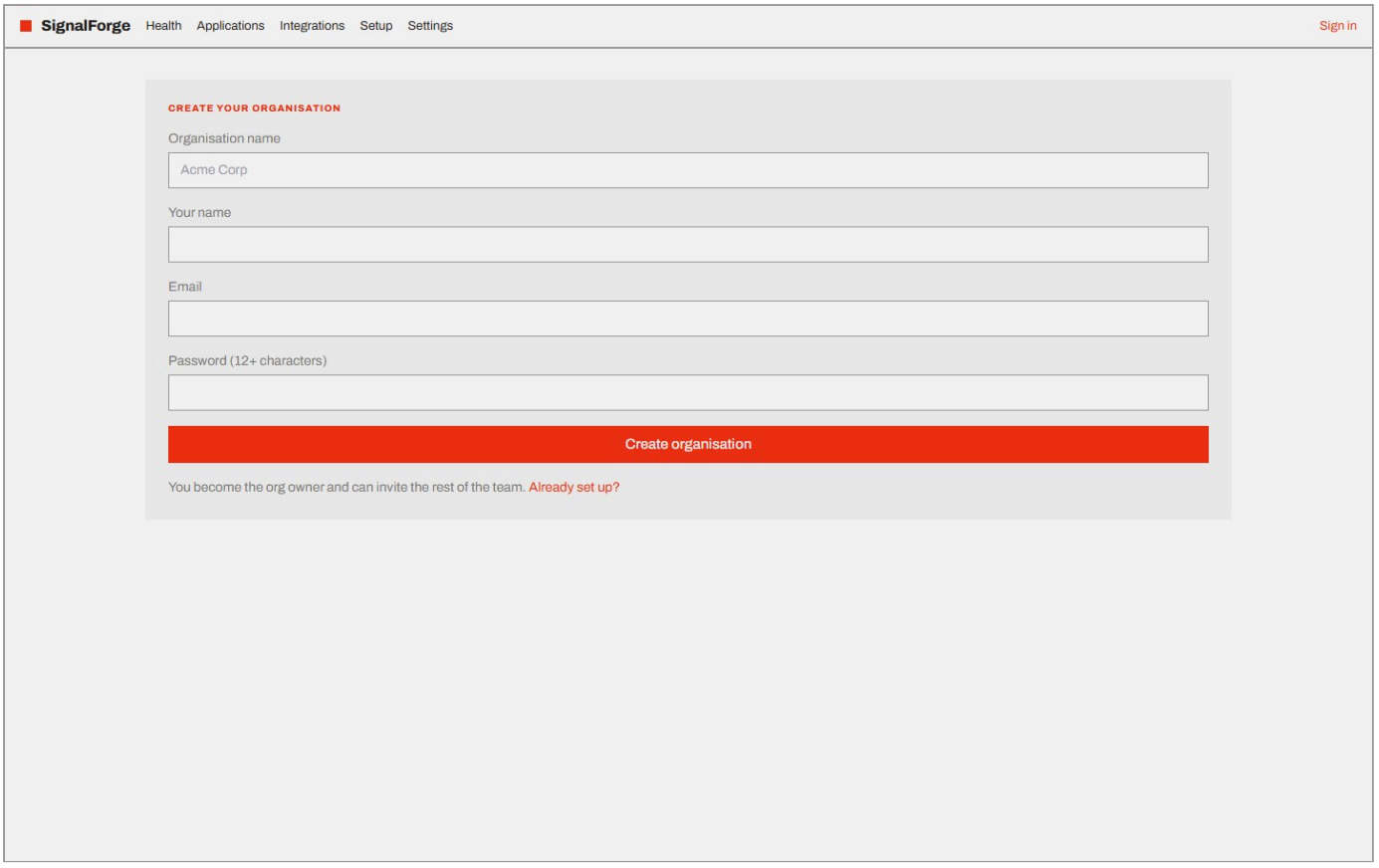
S1 Create an organisation

`/signup`

BUILT · VERIFIED LIVE

The first person to sign up creates the organisation and becomes its Org Owner. Everybody else arrives by invitation with roles attached — there is no second way in.

> **IN THIS CAPTURE**
> The empty form.



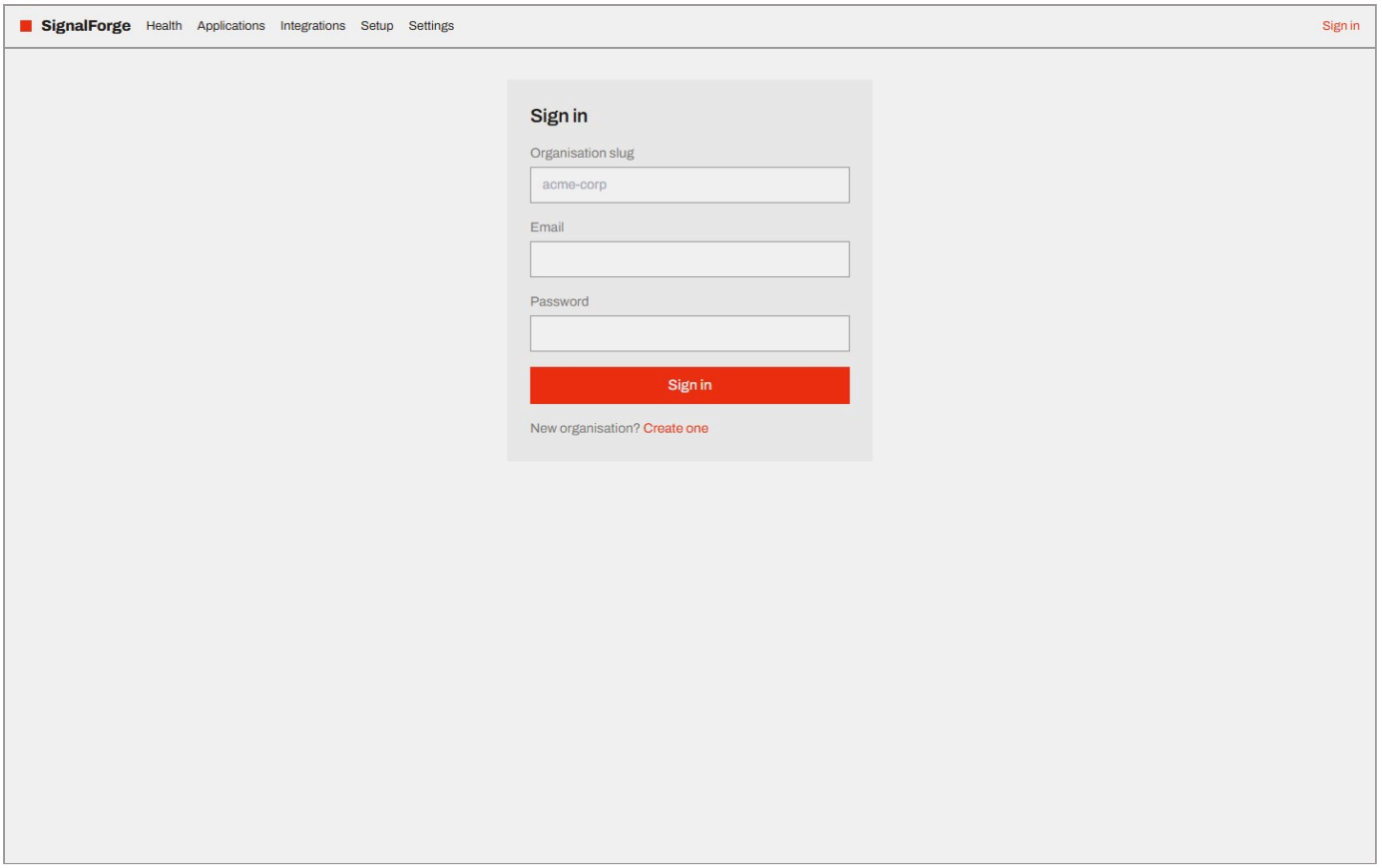
S1 Sign in

`/login`

BUILT · VERIFIED LIVE

Organisation slug, email, password. The slug is part of the credential rather than a picker after the fact, so a session can never be pointed at a second tenant.

> **IN THIS CAPTURE**
> The empty form.



S2 Organisation setup

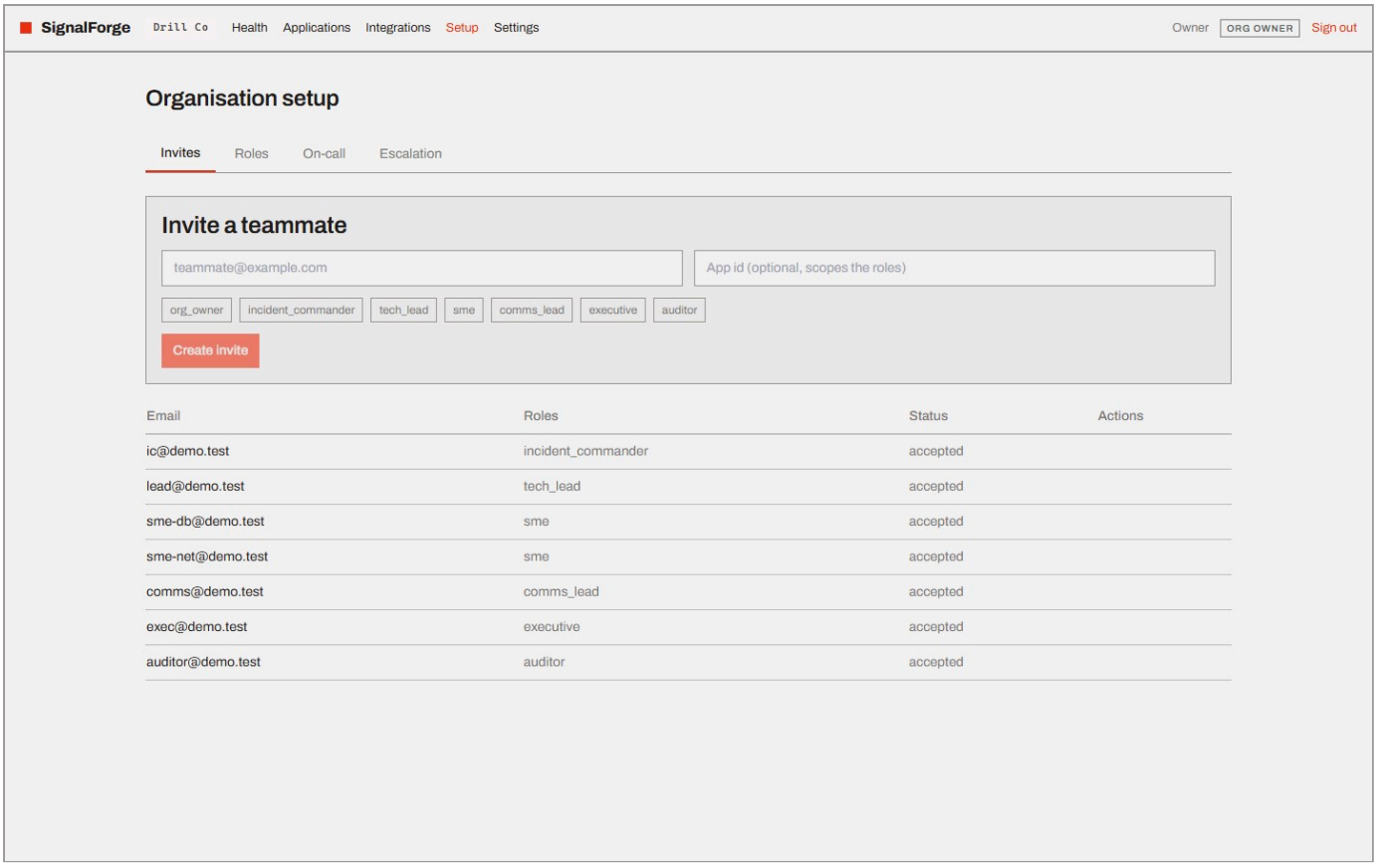
`/setup`

BUILT · VERIFIED LIVE

Four tabs: invites, the role matrix, on-call rotations and escalation policies. Roles can be scoped to a single application. The SME role deliberately holds no permissions — answering a page is self-scoped, so the page and the session authorise it, not a role.

IN THIS CAPTURE

Seven invitations accepted across all six non-owner roles: incident commander, tech lead, two SMEs, comms lead, executive and auditor.



SECTION B

Onboarding an application

> An application onboards through public contracts only. It serves twelve /observe/* paths and, if it wants to push, speaks the ingest contract with an HMAC. That is the whole integration requirement. The wizard is seven steps and is re-entrant — every step can be revisited without losing what the others established.

S3.1 Wizard — Basics

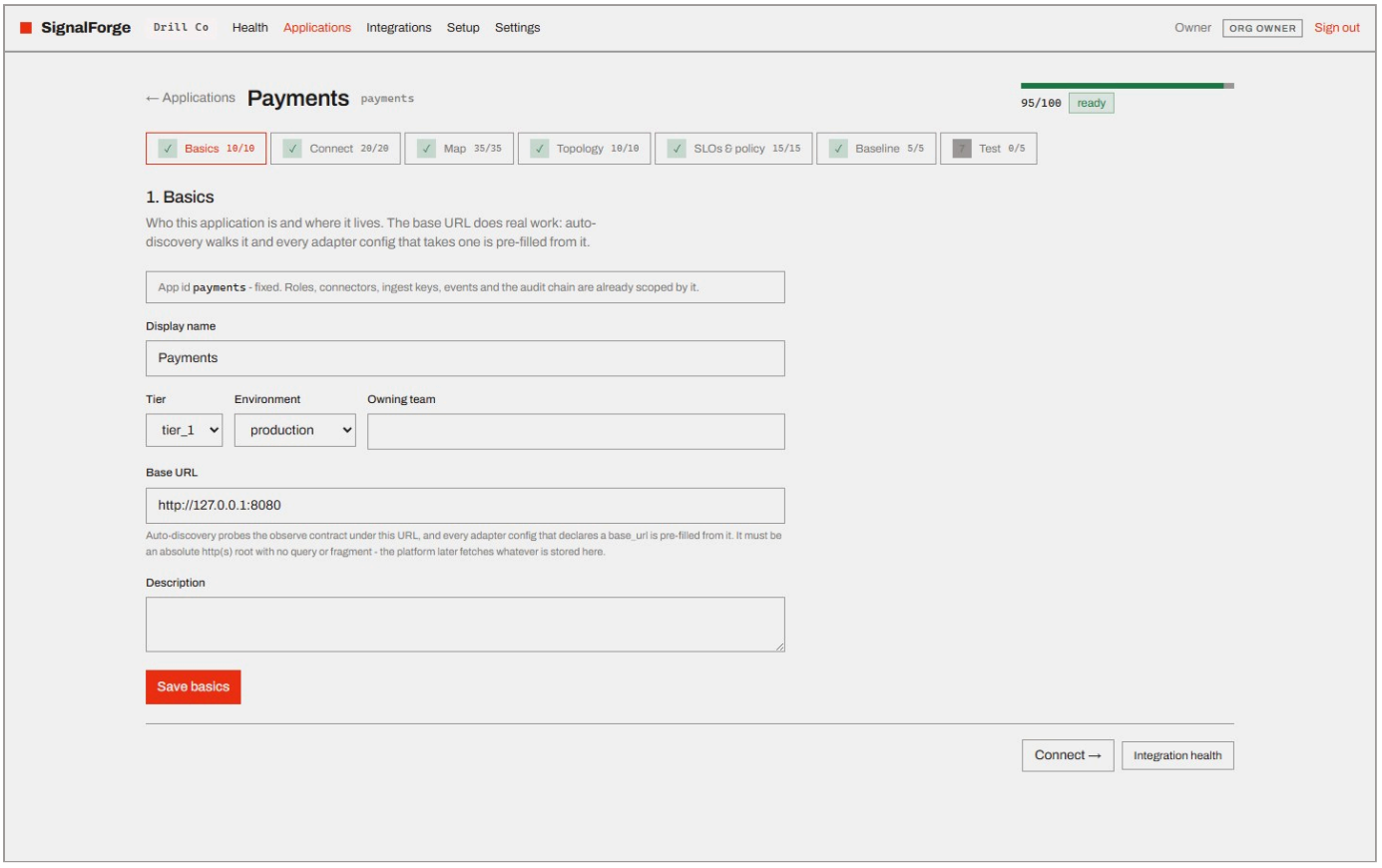
`/apps/{app}/onboard/1`

BUILT · VERIFIED LIVE

Who the application is and where it lives. The base URL does real work: auto-discovery walks it, and every adapter config that takes a URL is pre-filled from it.

IN THIS CAPTURE

The payments application, tier and environment set, base URL bound.



S3.2 Wizard — Connect

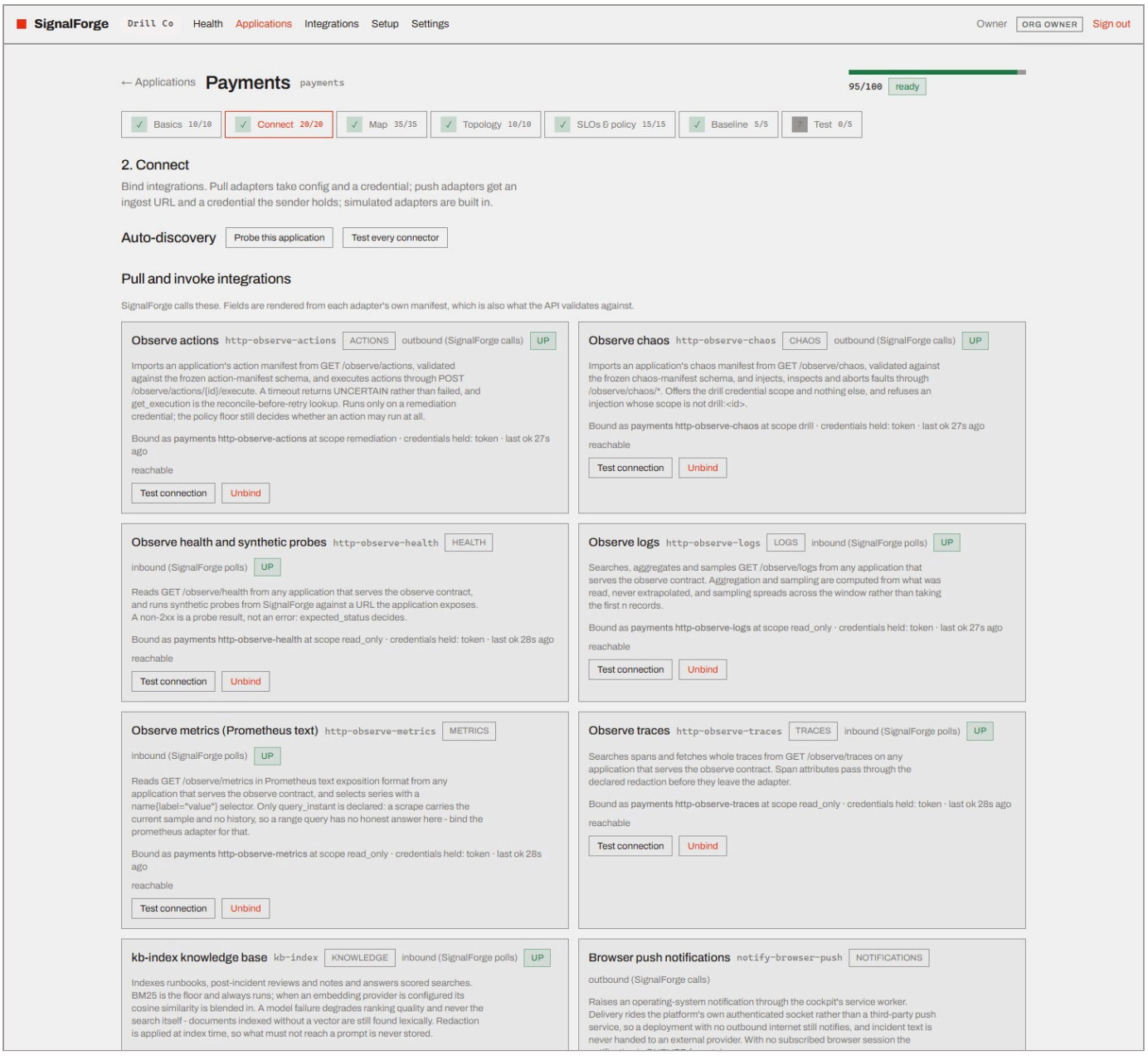
`/apps/{app}/onboard/2`

BUILT · VERIFIED LIVE

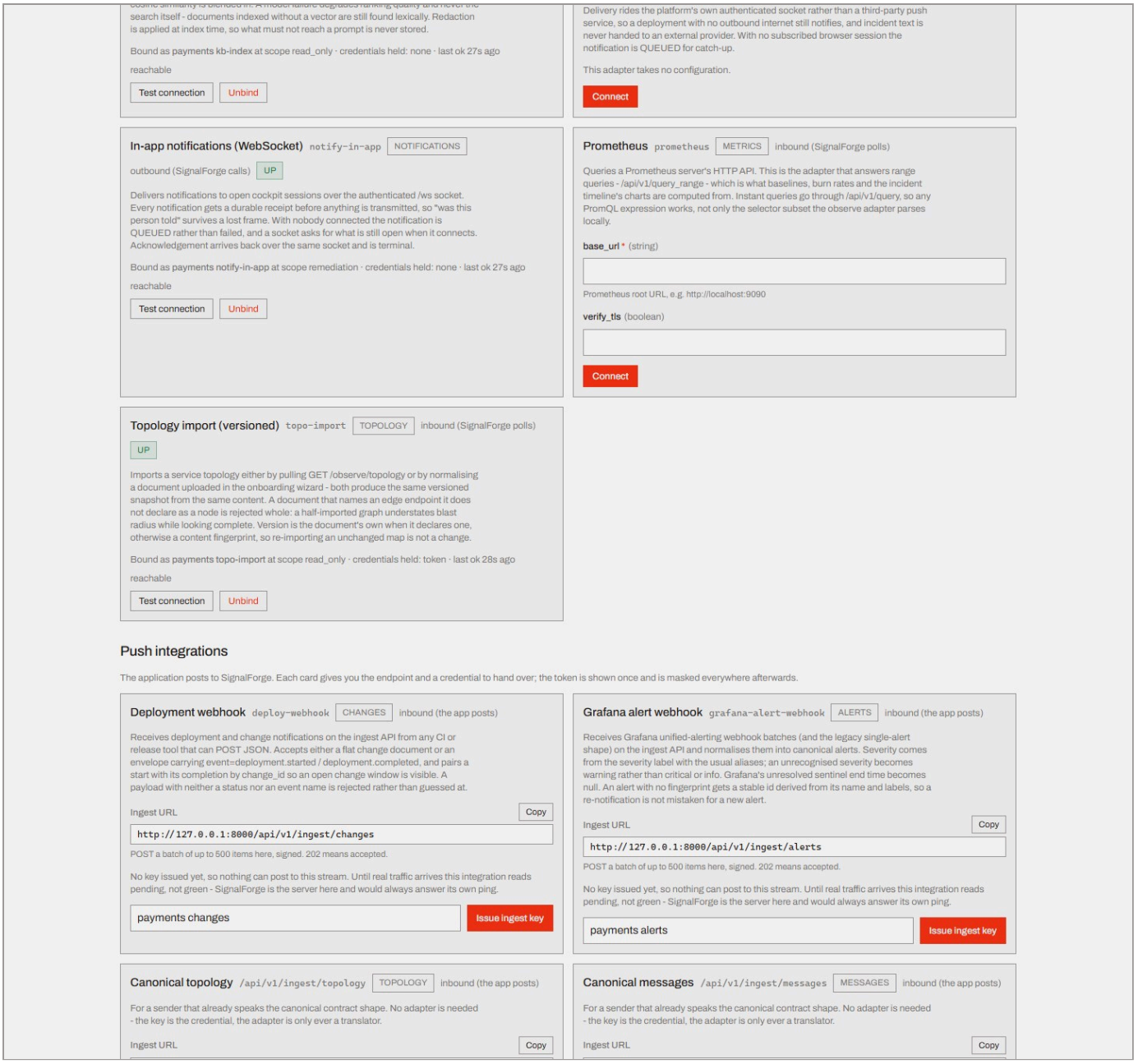
Bind integrations. Pull adapters take config and a credential; push adapters are handed an ingest URL and a credential the sender holds; simulated adapters are built in and are labelled SIMULATED everywhere they appear. Credentials are encrypted at rest and never read back.

IN THIS CAPTURE

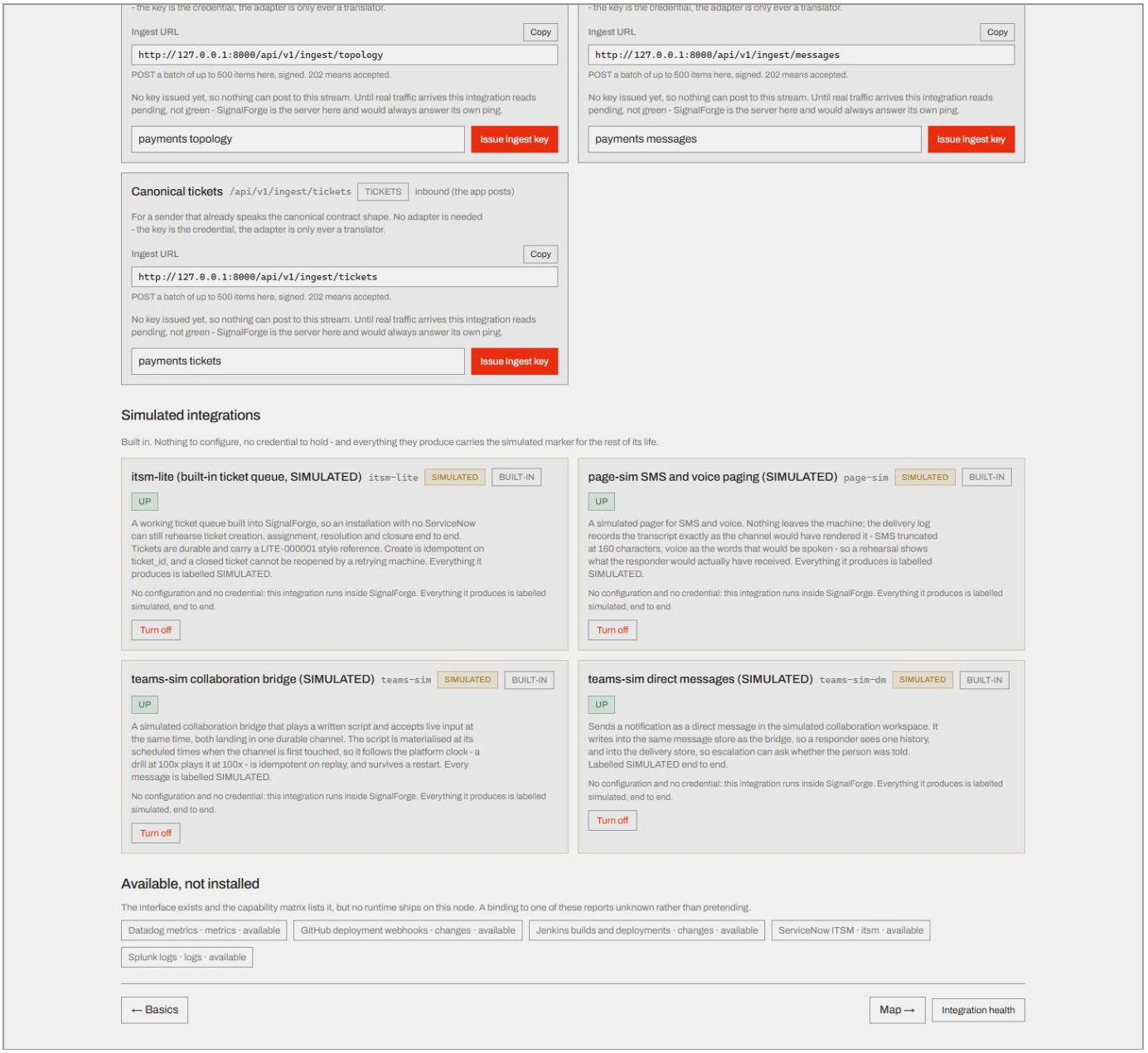
Fifteen connectors bound across the observe set, actions, chaos, ITSM, knowledge, notifications and collaboration, each with its own config and credential scope.



S3.2 · WIZARD — CONNECT — 1 OF 3



S3.2 · WIZARD — CONNECT — 2 OF 3



S3.2 · WIZARD — CONNECT — 3 OF 3

S3.3 Wizard — Map

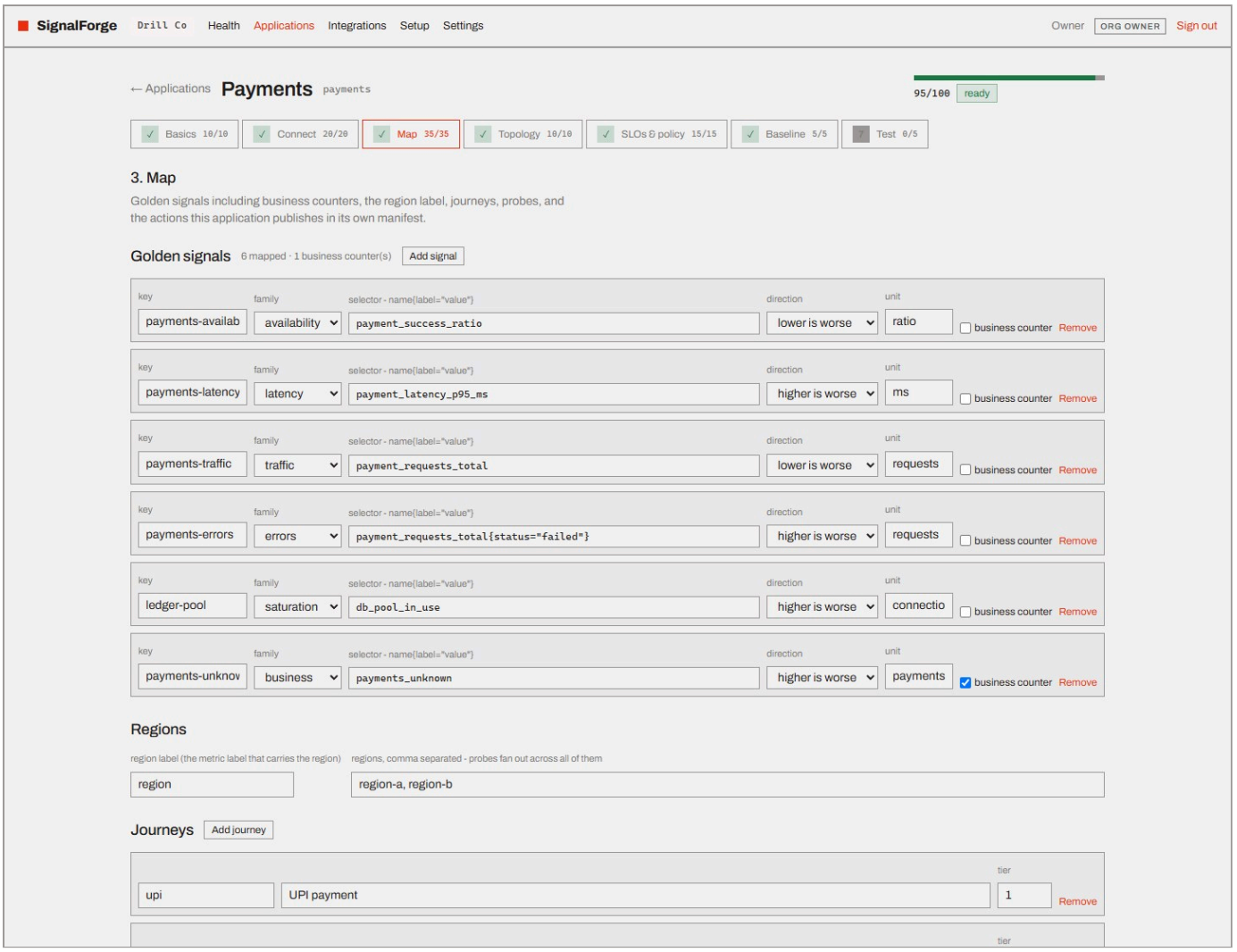
`/apps/{app}/onboard/3`

BUILT · VERIFIED LIVE

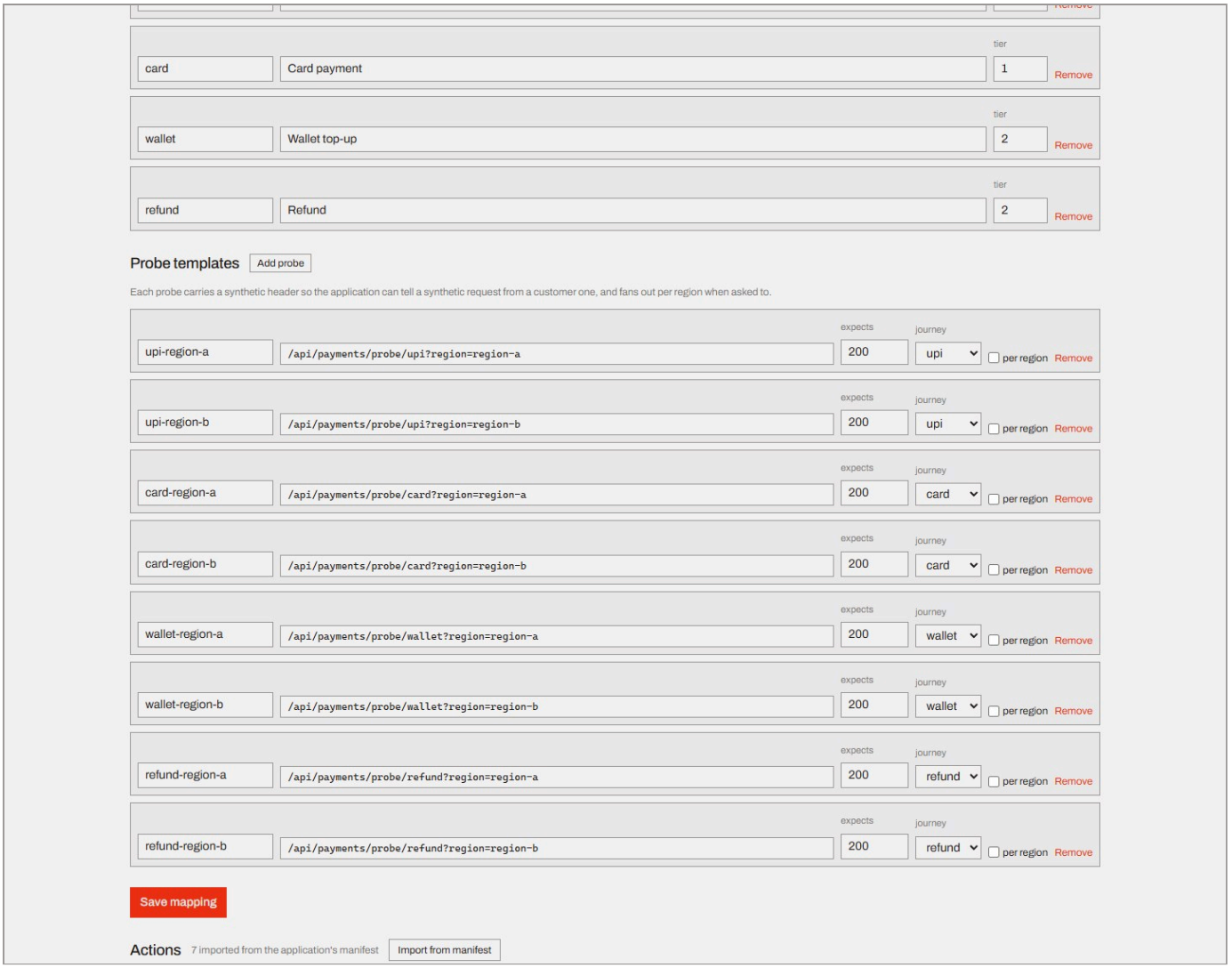
Golden signals including business counters, the region label, journeys and their probes, and the actions the application publishes in its own manifest with the risk class it declared. SignalForge does not invent an action.

IN THIS CAPTURE

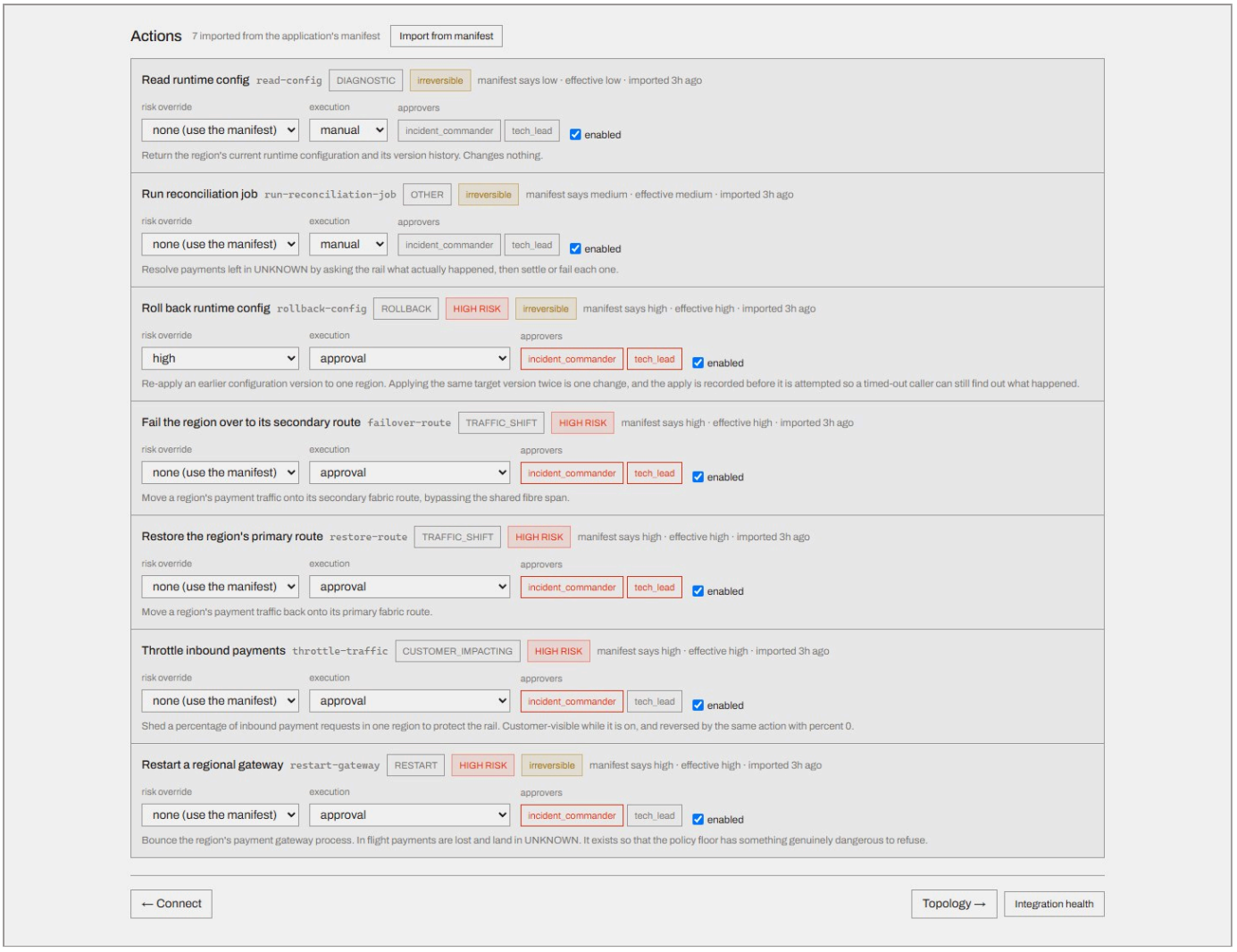
All six golden-signal families mapped to six real series, four journeys with a probe per region, and the imported action manifest with its risk classes.



S3.3 · WIZARD — MAP — 1 OF 3



S3.3 · WIZARD — MAP — 2 OF 3



S3.3 · WIZARD — MAP — 3 OF 3

S3.4 Wizard — Topology

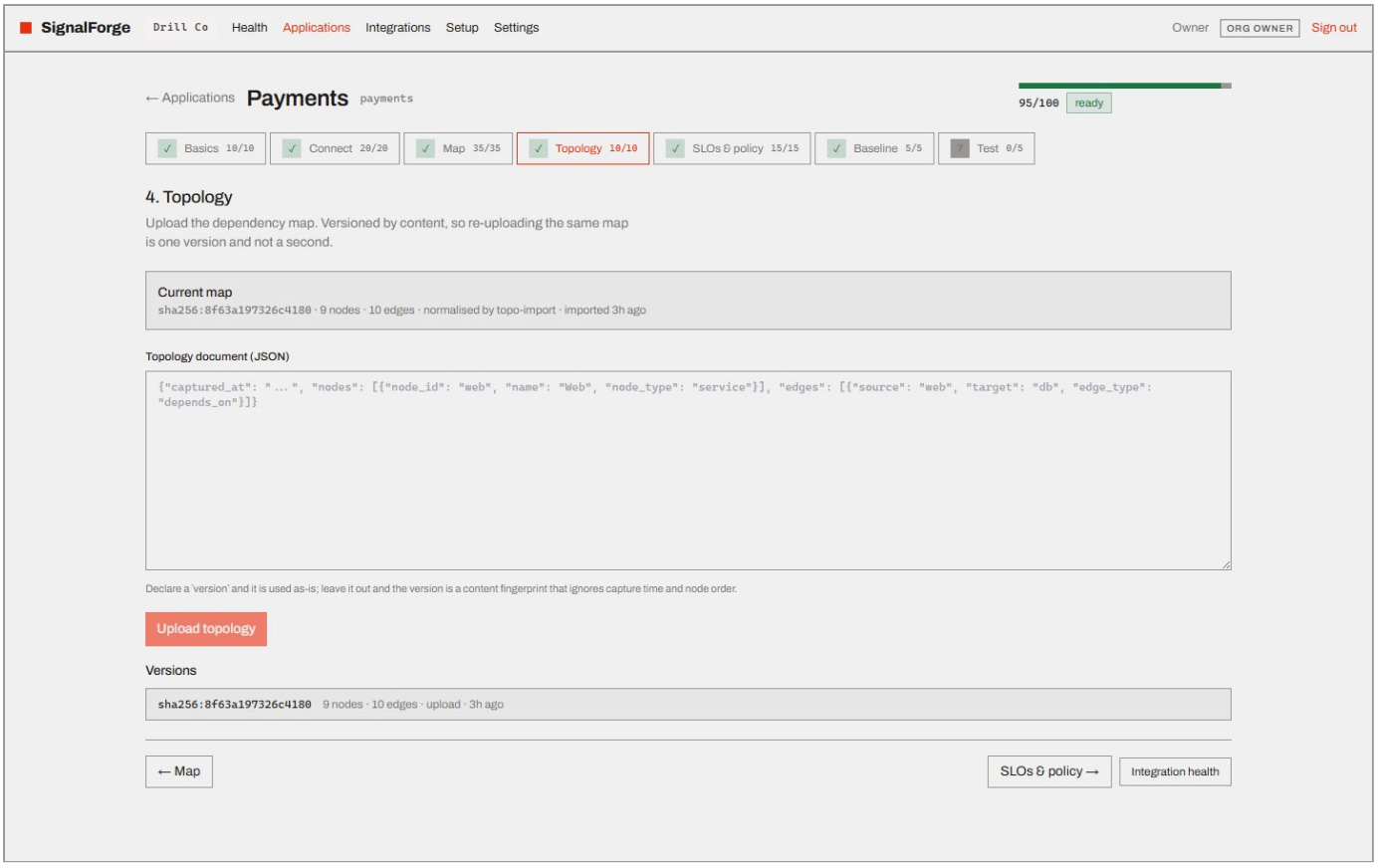
BUILT · VERIFIED LIVE

`/apps/{app}/onboard/4`

Upload the dependency map. Versions are keyed by content, so re-uploading the same map is one version and not a second, and a later different map produces a delta.

IN THIS CAPTURE

One version captured from the application's own routing table.



S3.5 Wizard — SLOs and policy

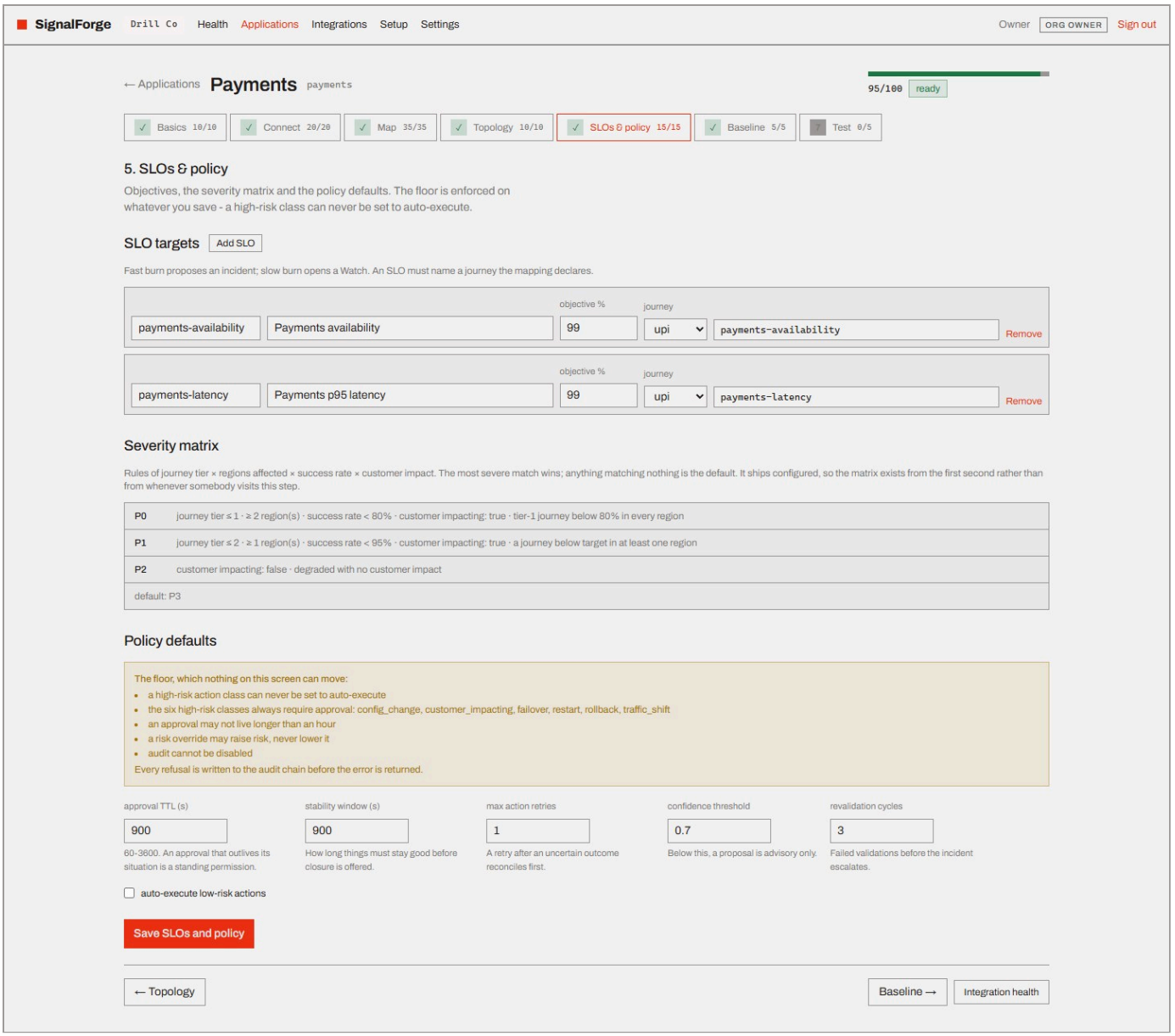
`/apps/{app}/onboard/5`

BUILT · VERIFIED LIVE

Objectives, the severity matrix and the execution-policy defaults. The policy floor is enforced at the point somebody types it: a high-risk class can never be saved as auto-execute, and audit can never be switched off. The rejection is counted in a Prometheus series.

IN THIS CAPTURE

Two SLOs and the severity matrix, above the floor.



S3.6 Wizard — Baseline

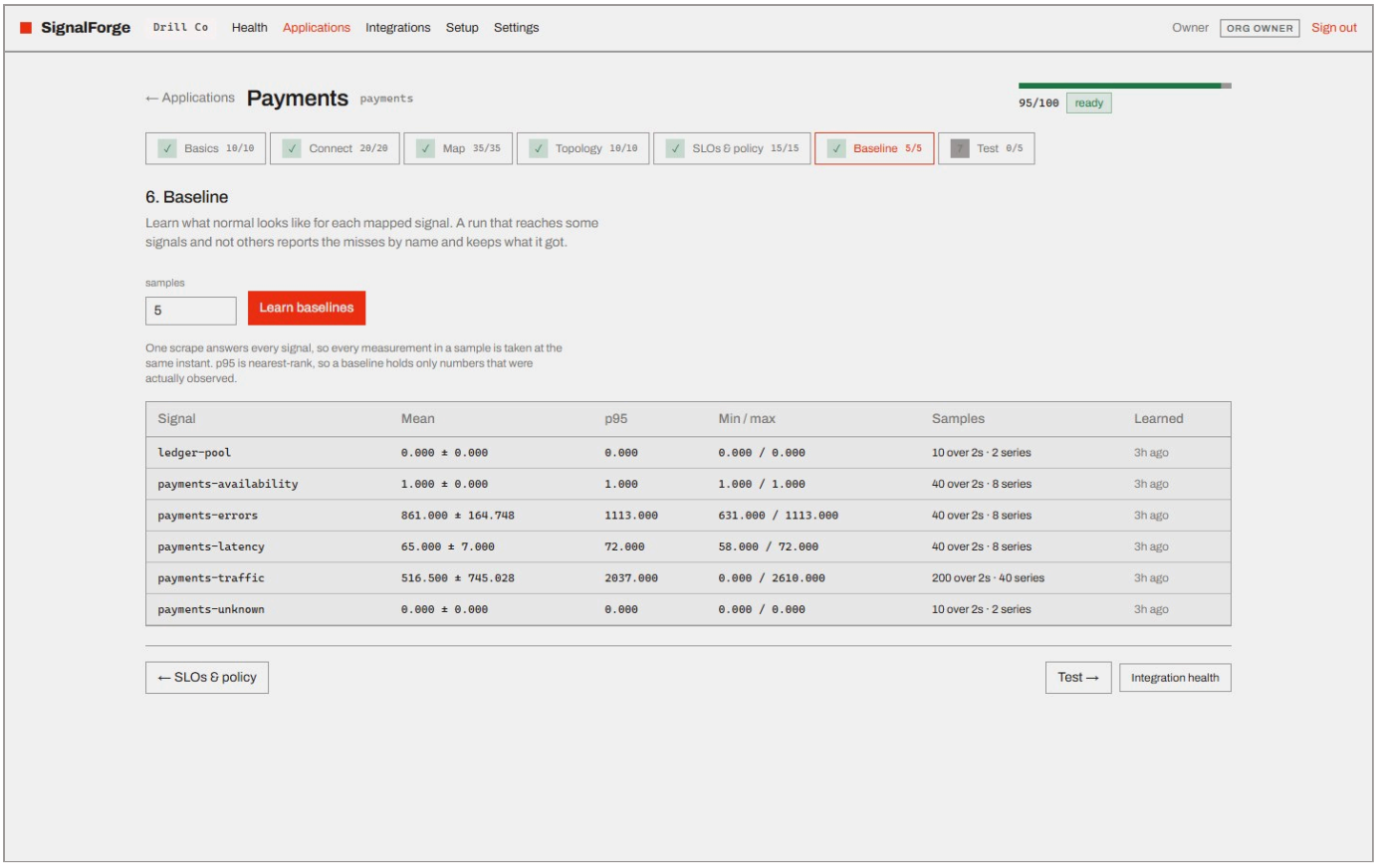
`/apps/{app}/onboard/6`

BUILT · VERIFIED LIVE

Learn what normal looks like for each mapped signal. A run that reaches some signals and not others is a partial success: it reports the misses by name and keeps what it got.

IN THIS CAPTURE

Baselines formed for the mapped signals.



S3.7 Wizard — Test

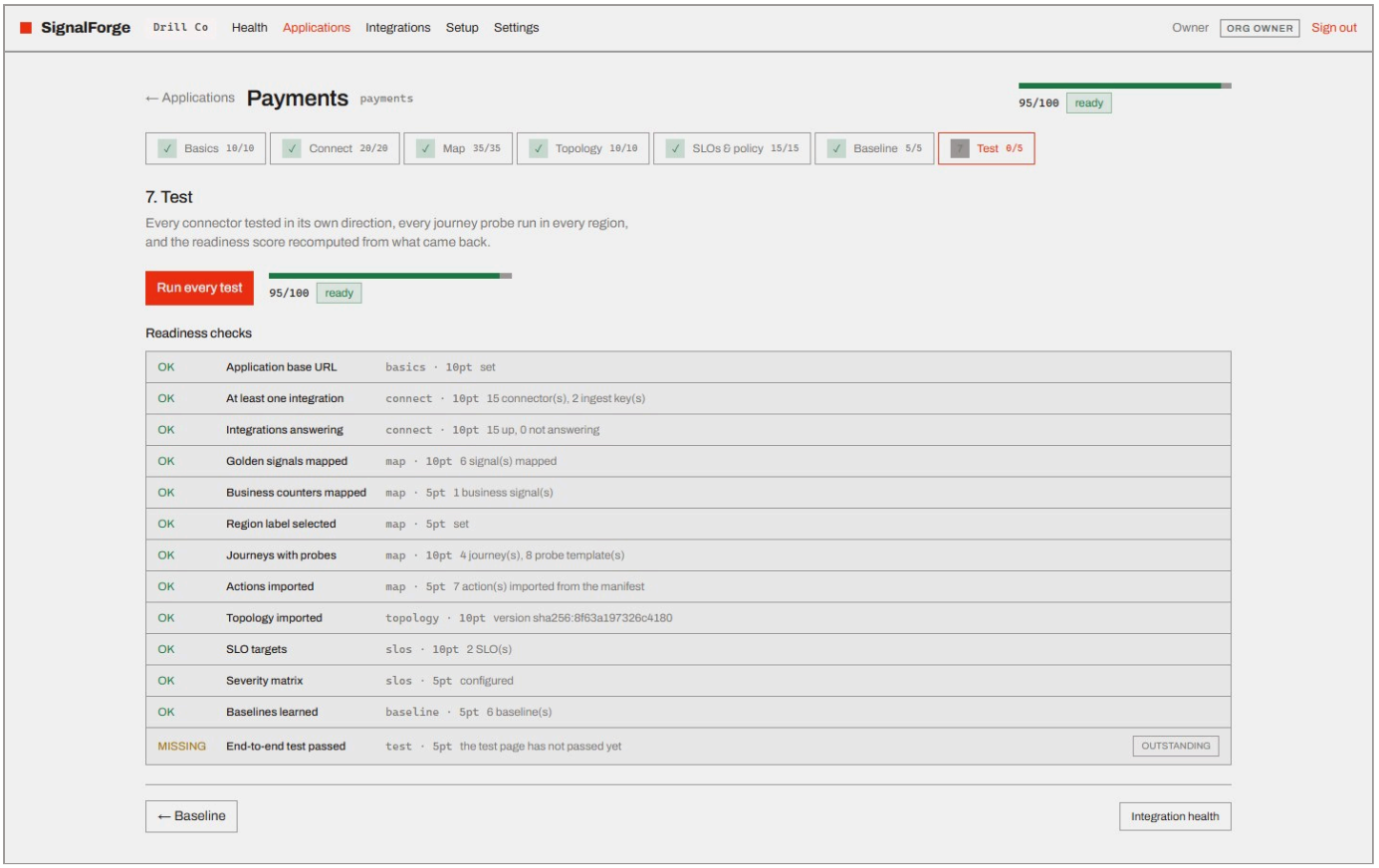
`/apps/{app}/onboard/7`

BUILT · VERIFIED LIVE

Every connector tested in its own direction — a push connector is not reachability-checked, because SignalForge is the server for it — every journey probe run in every region, and the readiness score recomputed from what came back. A failing test is a normal outcome and the wizard shows what the application answered.

IN THIS CAPTURE

The readiness panel with the score recomputed.



S4 Applications

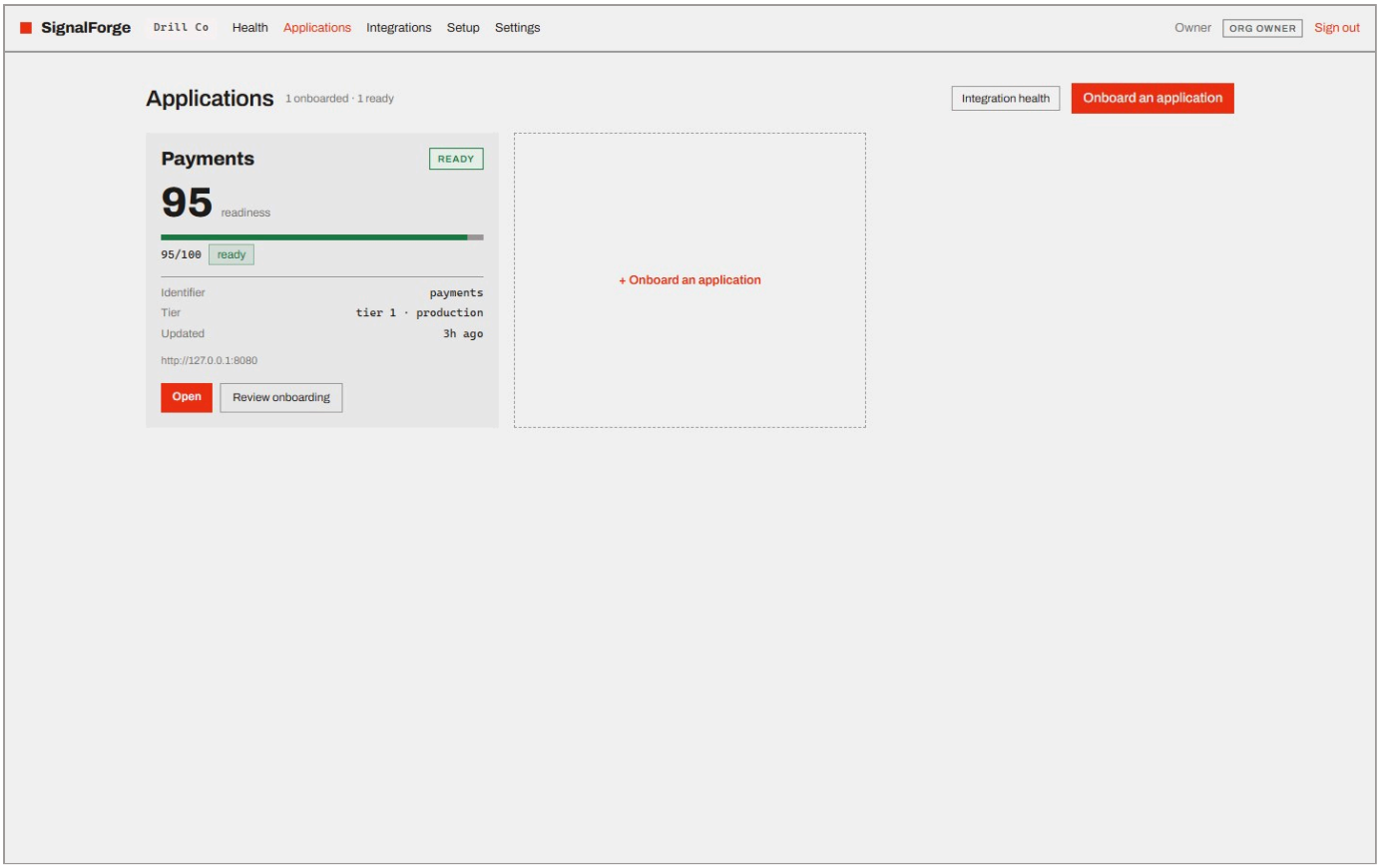
/apps

BUILT · VERIFIED LIVE

The estate as cards, each with its readiness and its current stance. New applications start here.

IN THIS CAPTURE

One onboarded application in this organisation. A second application (drills/second-app.yaml, deliberately shaped differently — four signals, two journeys, one region, no chaos, no topology) exists as data and is not part of this capture.



S23 Integration health

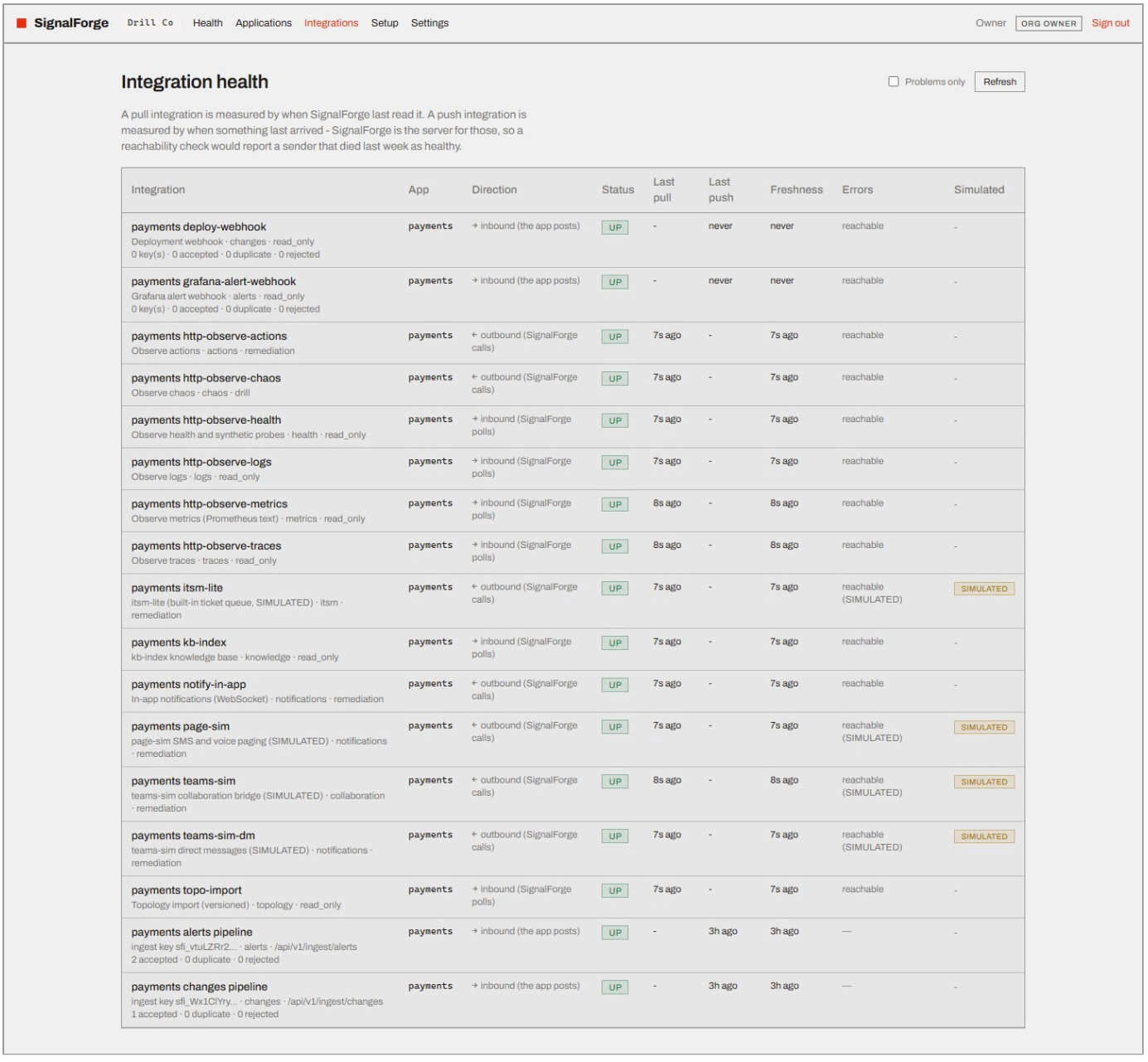
`/integration-health`

BUILT · VERIFIED LIVE

Every connector, measured the right way round. A pull integration is measured by when SignalForge last read it; a push integration by when something last arrived, because a reachability check would report a sender that died last week as healthy. A connector that stops answering is reported and does not take the cockpit with it.

IN THIS CAPTURE

Seventeen connectors, all UP, with direction, freshness, error state, per-key accepted/duplicate/rejected counts, and SIMULATED labelling on the four that are simulated.



SECTION C

Detection, and the decision to declare

> Detection is four jobs on a loop: scrape, roll up, baseline, evaluate. It produces a stance — Steady, Watch or Proposed — and a severity computed from measured impact. It never declares. Declaring is one of three acts reserved for people; the other two are approving a high-risk action and closing.

S5 An application

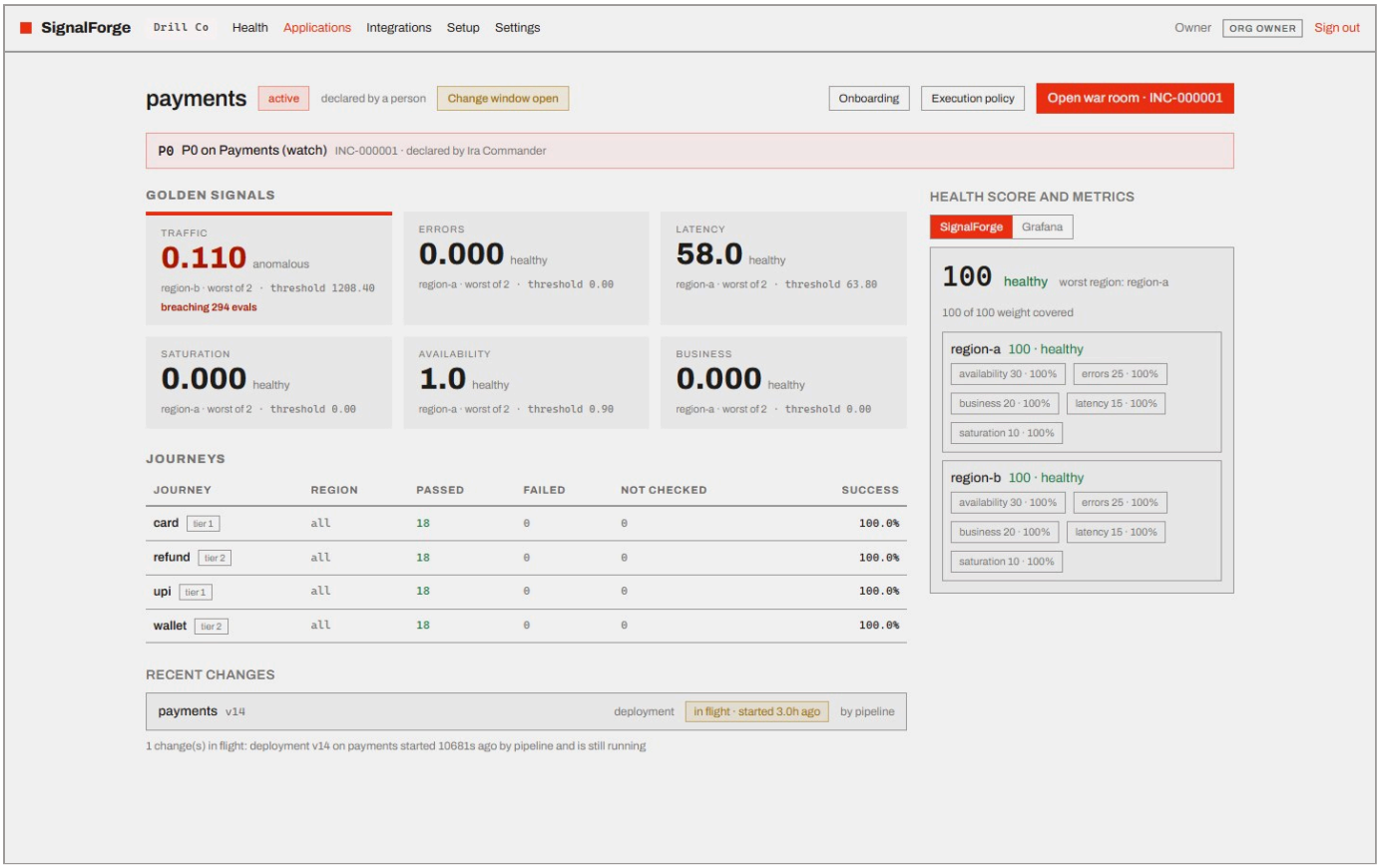
`/apps/{app}`

BUILT · VERIFIED LIVE

Six golden-signal tiles, each showing the reading, its threshold, the worst region and how long it has been breaching; journeys as a table with pass/fail per region; recent changes from the deploy hook; and the health score renormalised over the weight actually covered, worst region winning. This is the screen a proposal appears on and the screen a person declares from.

IN THIS CAPTURE

Traffic anomalous on region-b, breaching 294 evaluations, against five healthy families. Health 100 over two regions with 100 of 100 weight covered. Four journeys at 100%. A deployment in flight. A declared P0 already open.



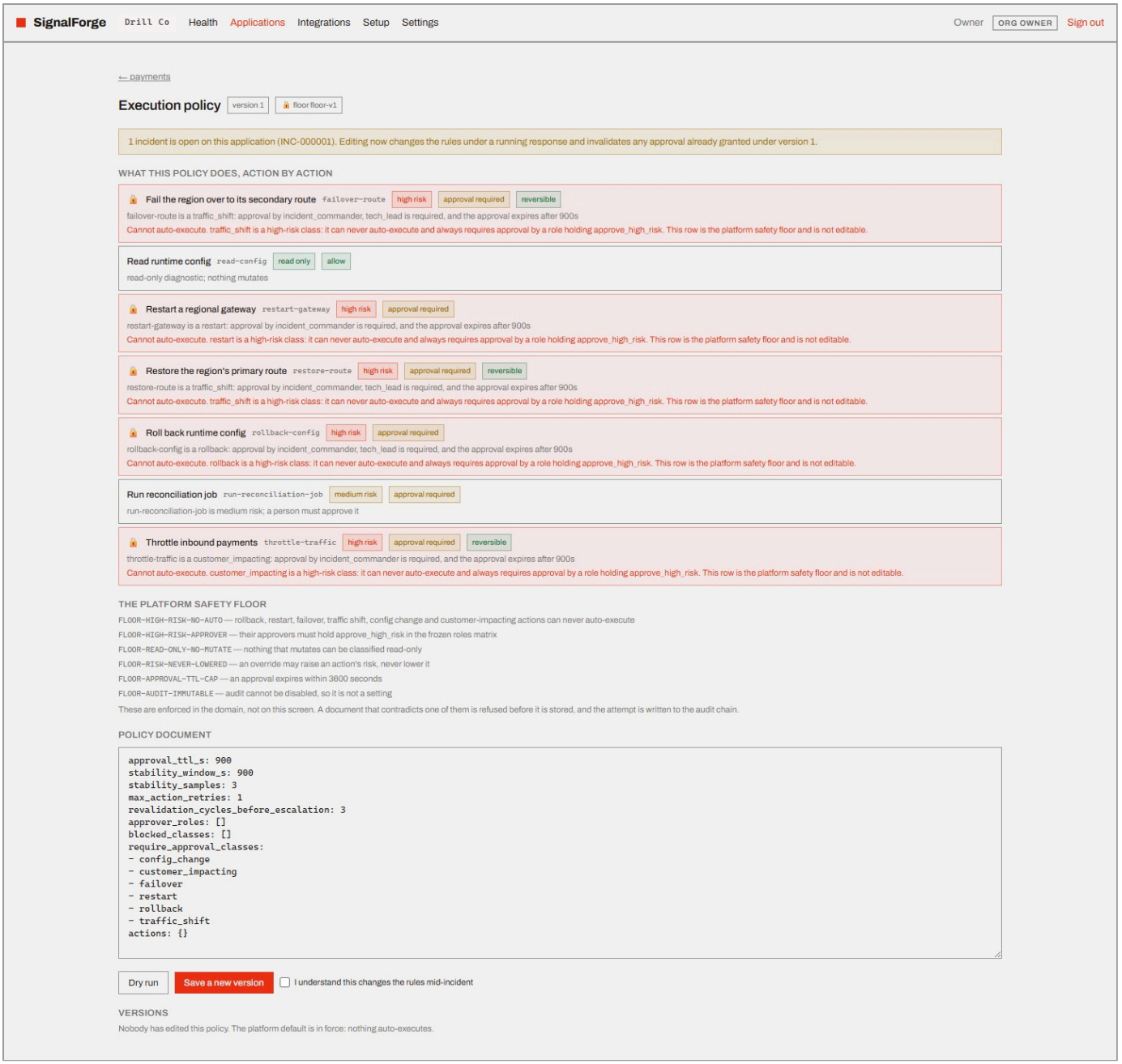
S21b Execution policy

`/apps/{app}/policy`

BUILT · VERIFIED LIVE

The per-application execution policy that sits on top of the hardcoded floor. Its own route rather than a tab, because editing a policy is a different act from watching an application and carries a different permission. Changes are versioned and can be dry-run before they are saved.

> **IN THIS CAPTURE**
> Policy v1 for payments, with the floor rules shown as unremovable.



S8 Incidents

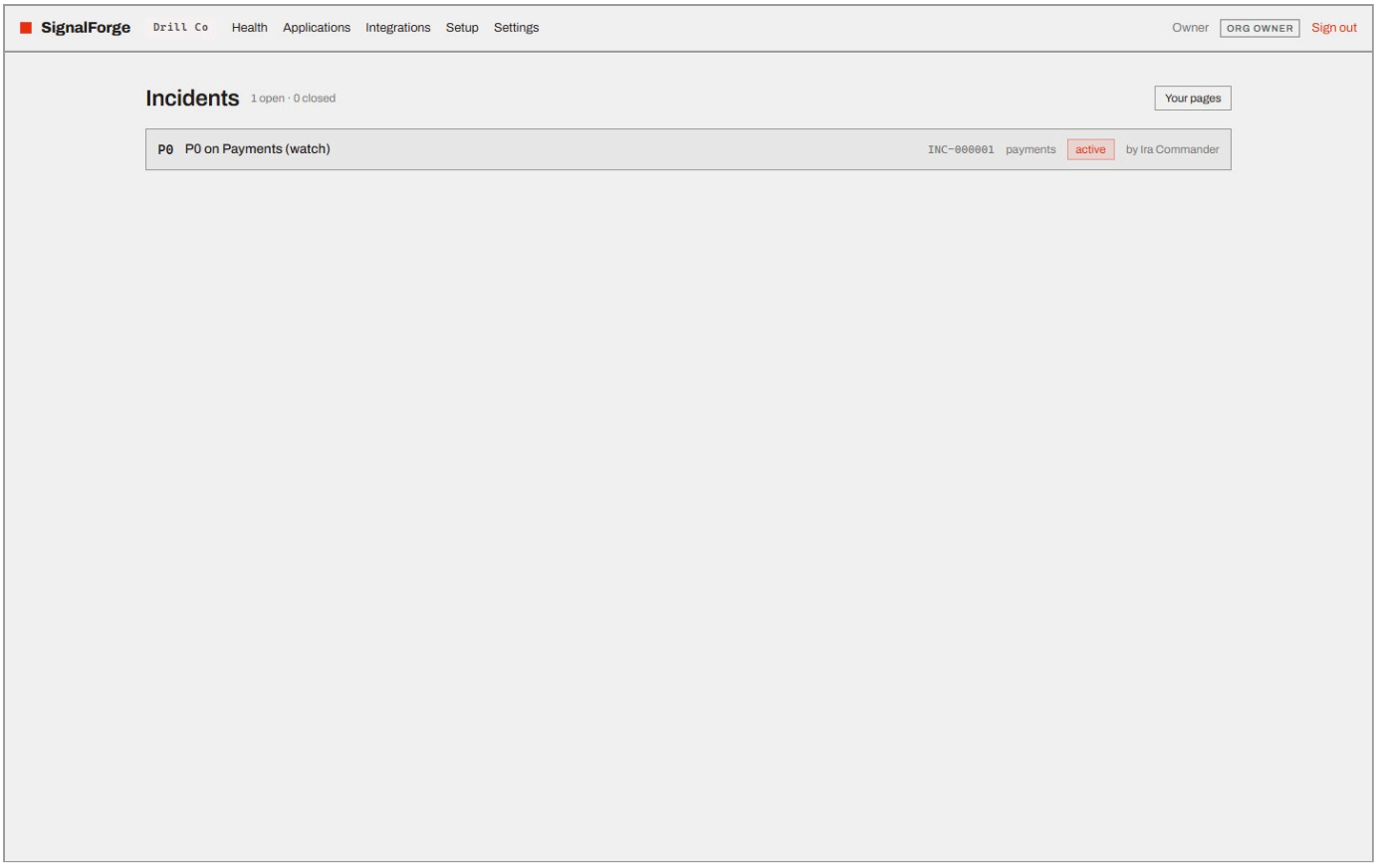
`/incidents`

BUILT · VERIFIED LIVE

Open and closed incidents for the organisation, with severity, application and who declared.

IN THIS CAPTURE

One open P0, declared by a person, zero closed.



SECTION D

The war room

> One screen, and its grid is the reading order of a war room: testimony on the left, decision in the middle, measurement on the right. A commander scanning left to right goes from what people are telling them, through what to do, to what is actually true. S10 to S18 are not ten screens — each is this screen with one panel expanded into a document, addressable as ?view=… so somebody can send a colleague a link to the thing they are talking about.

S9 The cockpit

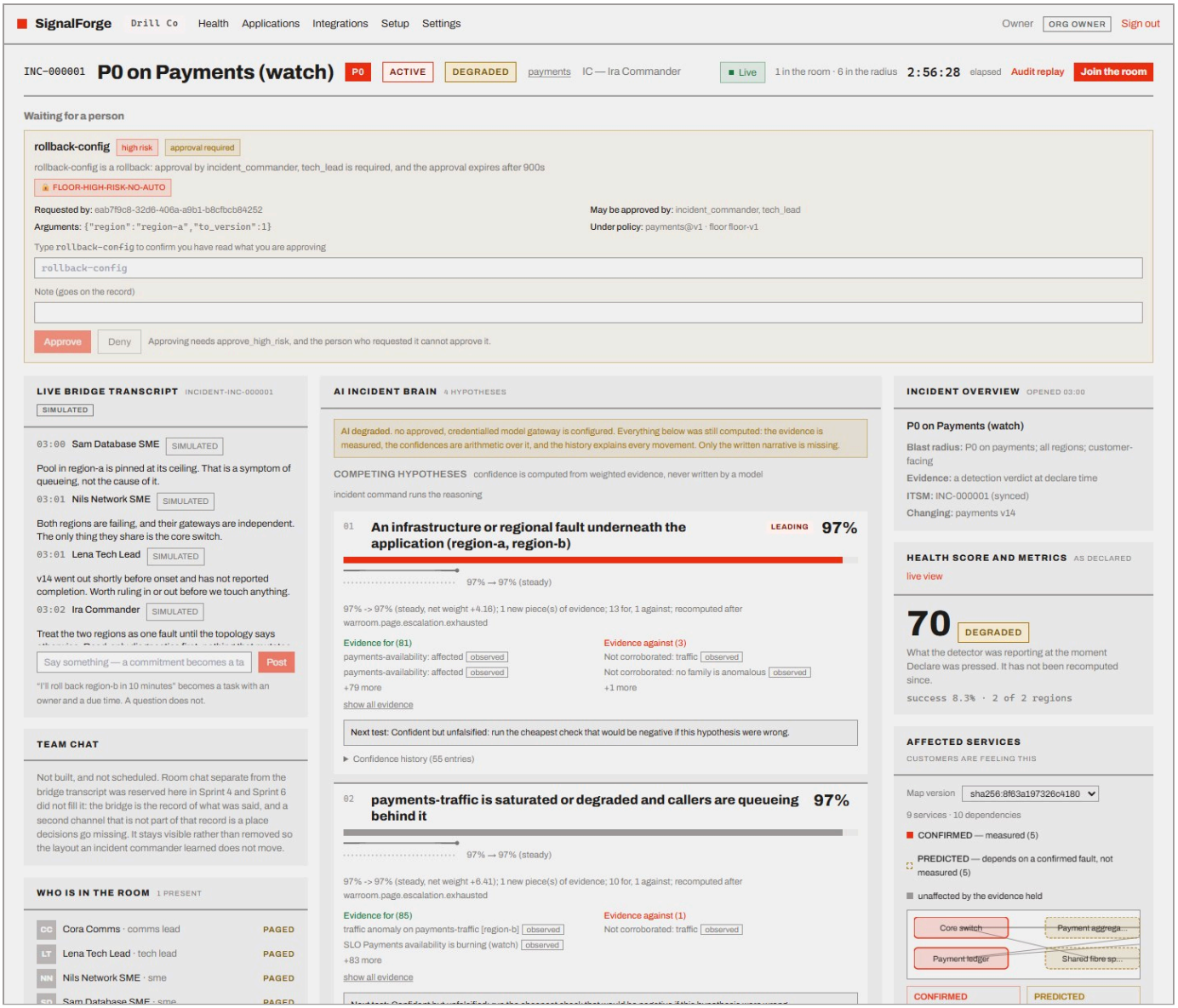
`/incidents/{id}`

BUILT · VERIFIED LIVE

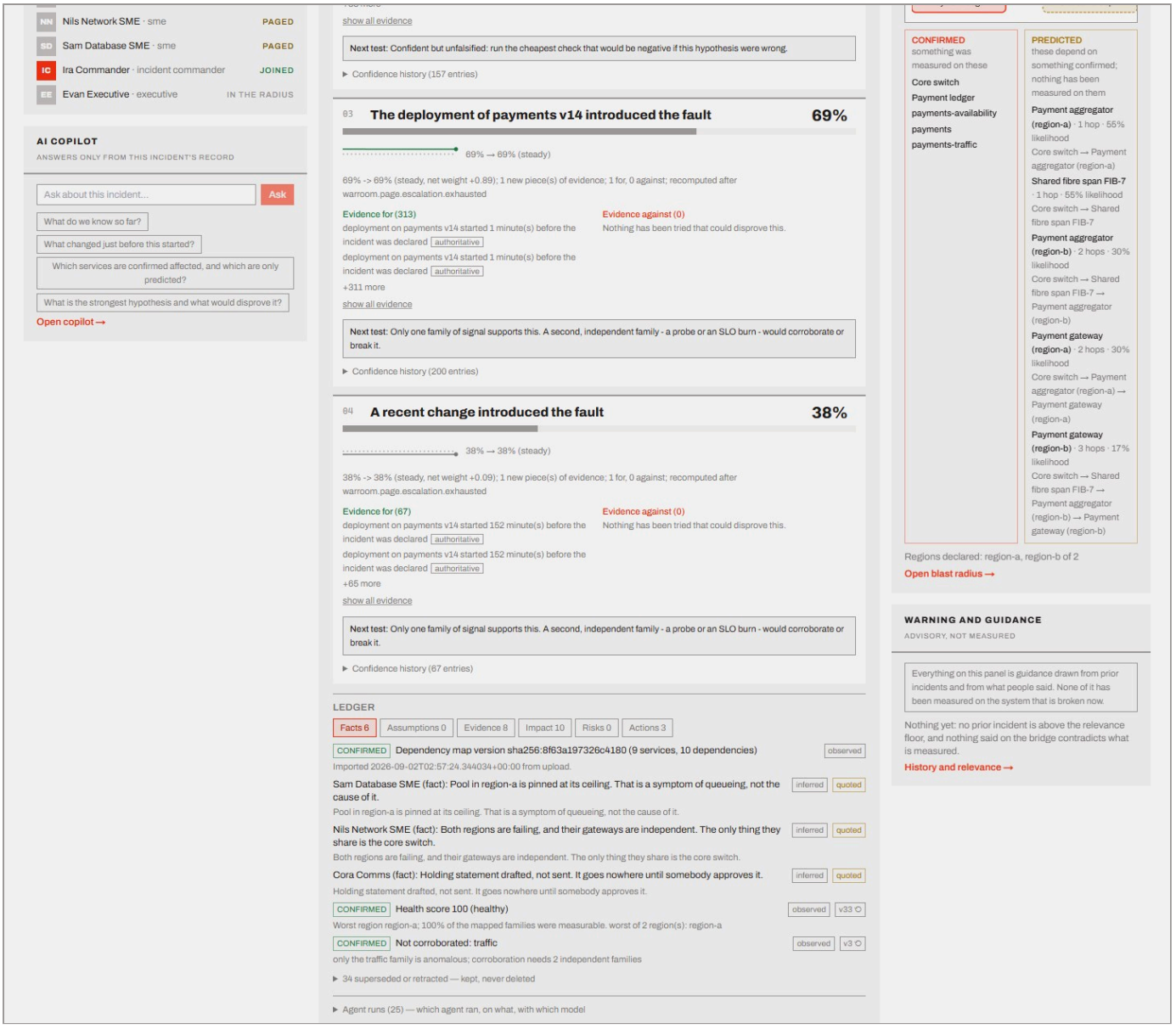
Ten panels. Left: the live bridge transcript, team chat, who is in the room with their acknowledgement state, and the copilot. Centre: the incident brain's competing hypotheses, the ledger, next best action, and the incident timeline. Right: the incident overview, health, affected services split into CONFIRMED and PREDICTED, and warning and guidance. Below: execution ledger, recovery validation, response plan, communications, the paging board and commitments.

IN THIS CAPTURE

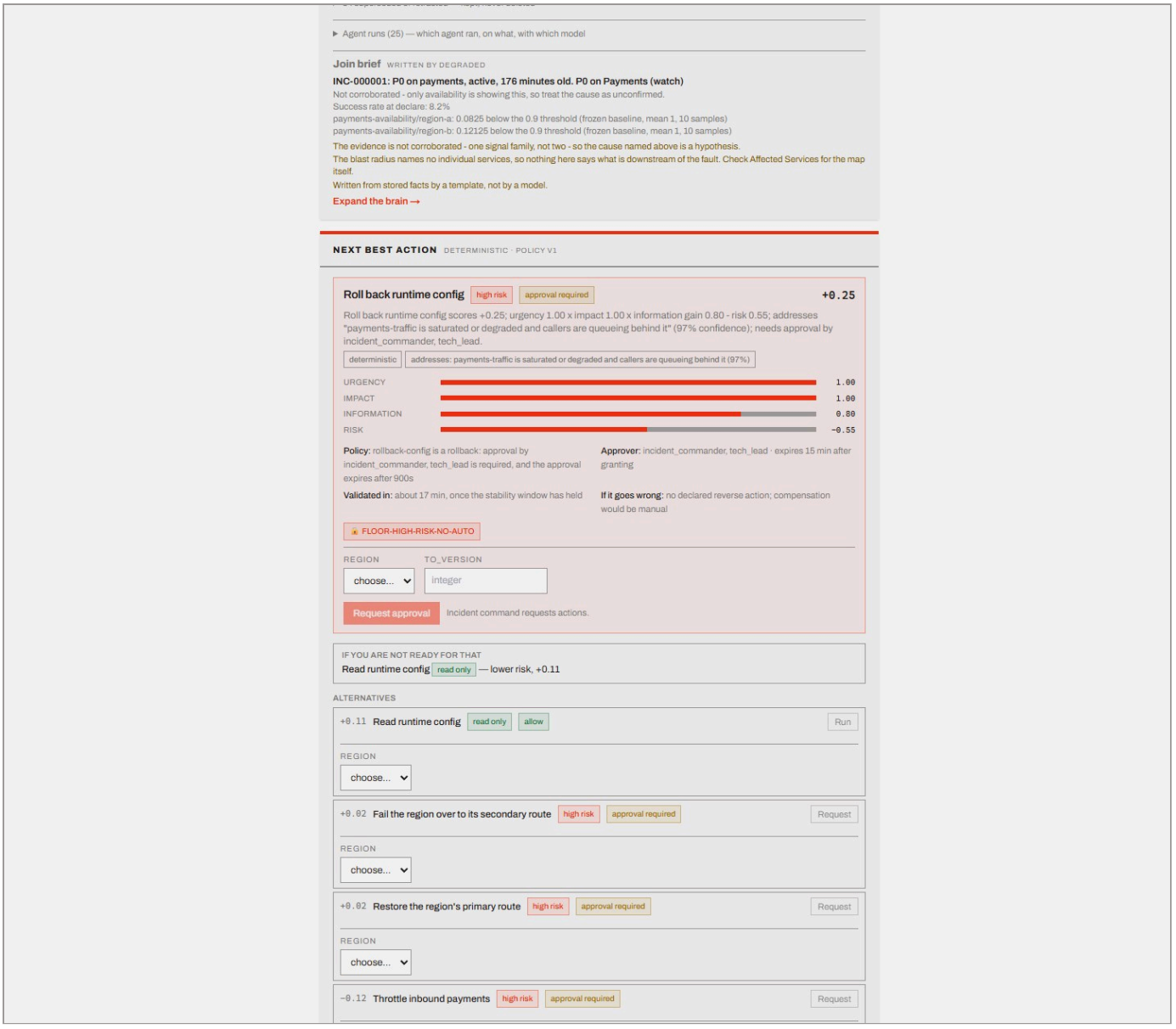
A live P0 at 2h56m elapsed, one person in the room of six in the radius, a high-risk rollback waiting for a second person to approve it, four competing hypotheses, a full paging board with escalations and overdue timers, and one bridge commitment turned into a task.



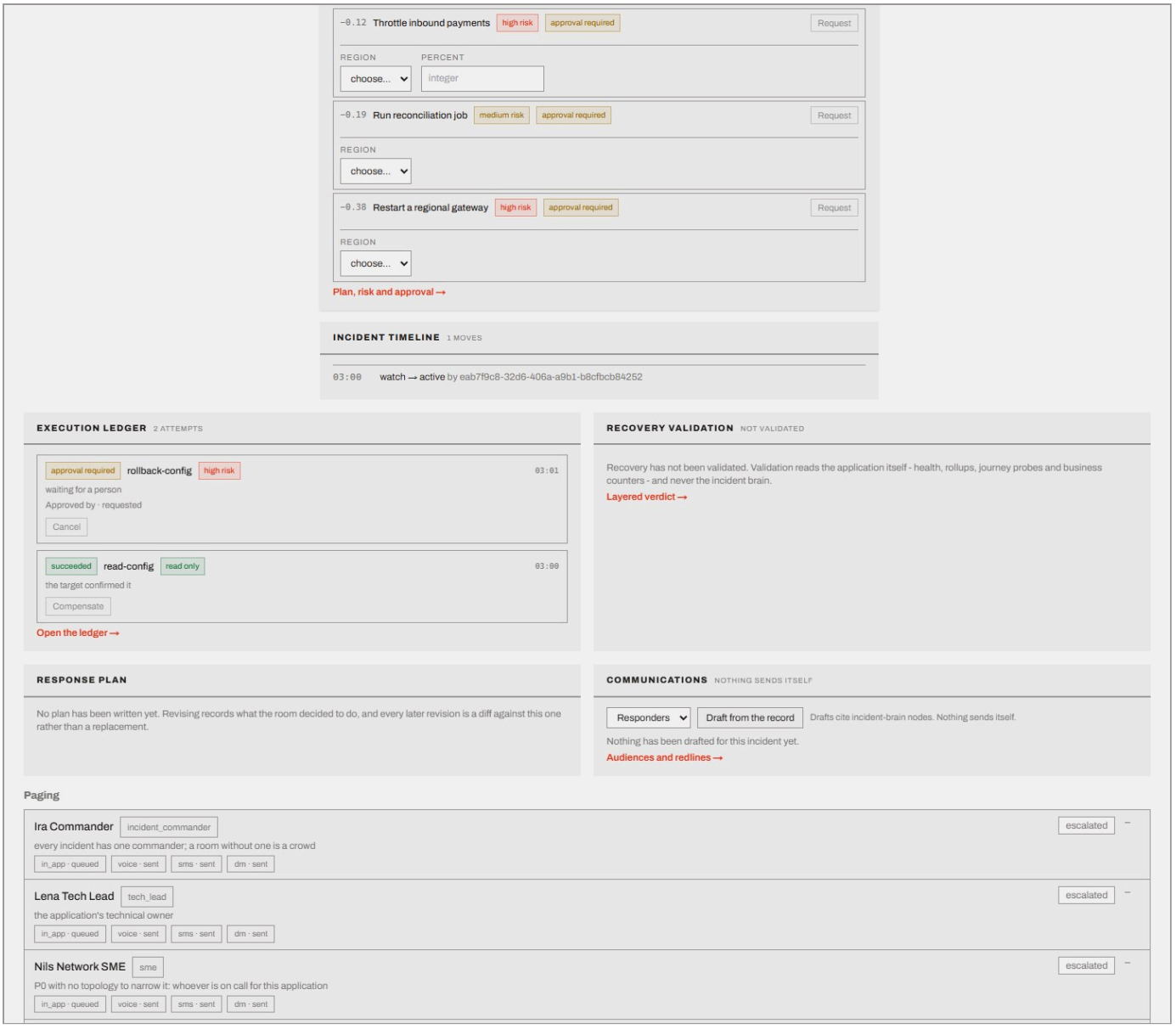
S9 · THE COCKPIT — 1 OF 5



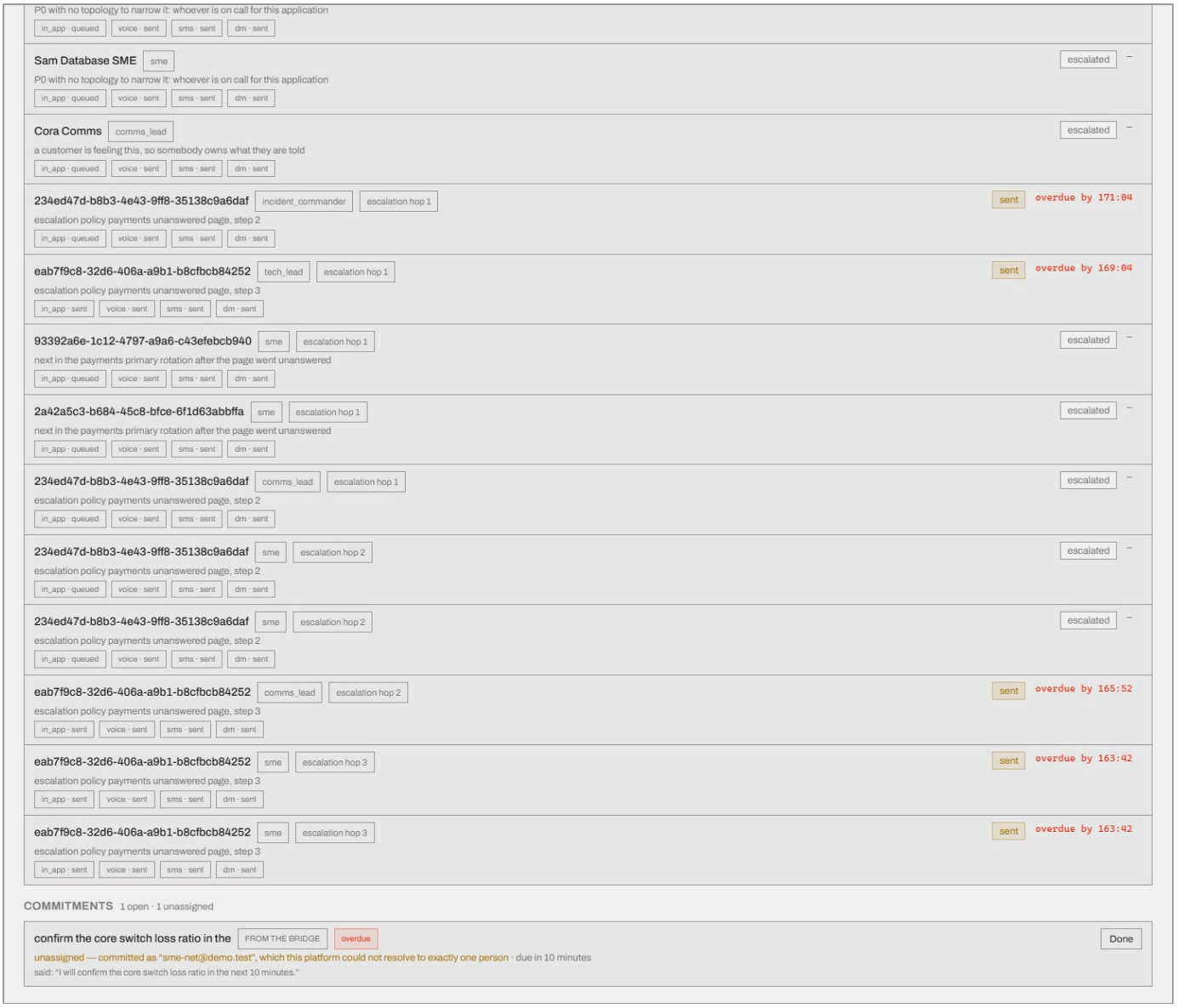
S9 · THE COCKPIT — 2 OF 5



S9 · THE COCKPIT — 3 OF 5



S9 · THE COCKPIT — 4 OF 5



S9 · THE COCKPIT — 5 OF 5

S10 Incident brain

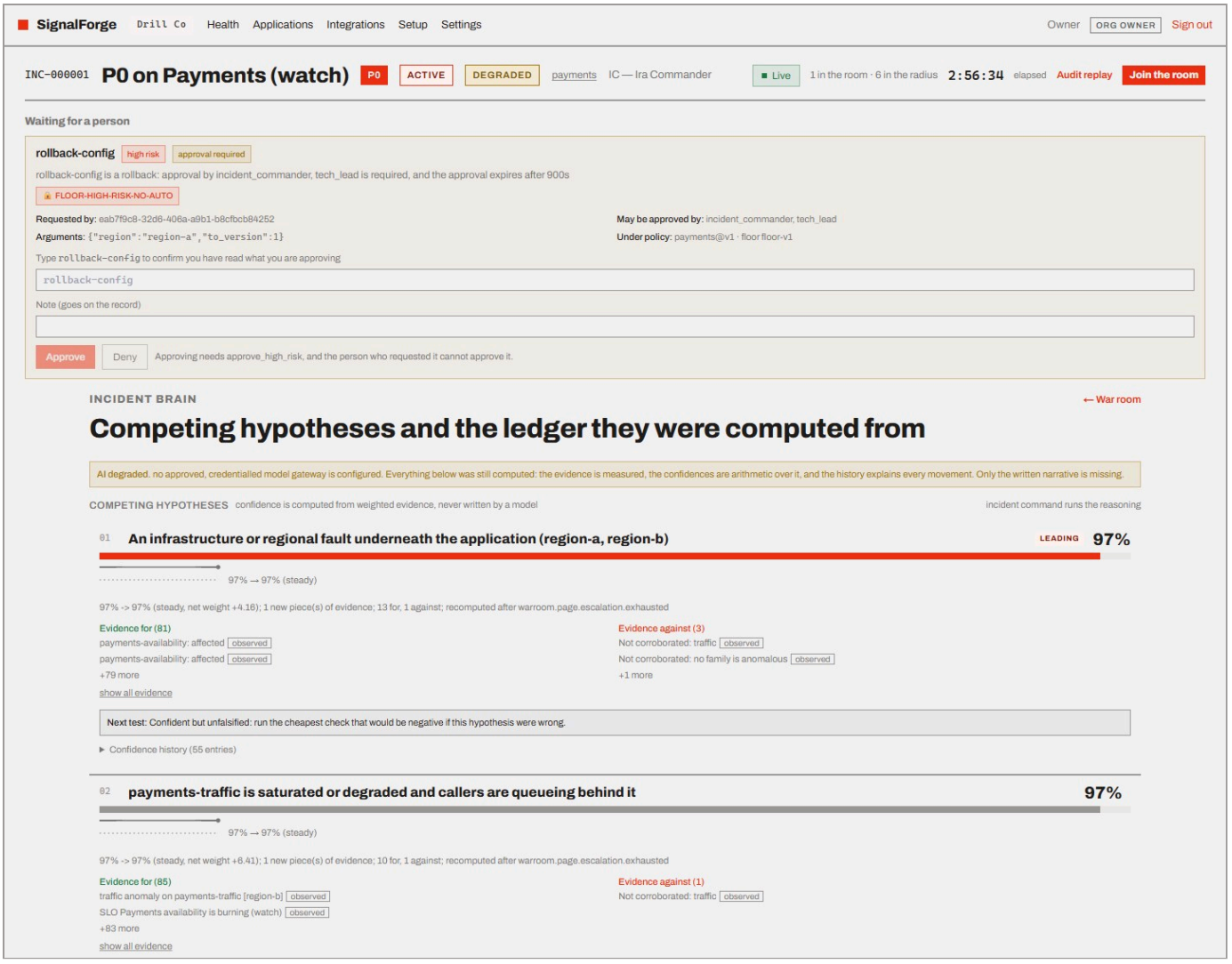
?view=brain

BUILT · VERIFIED LIVE

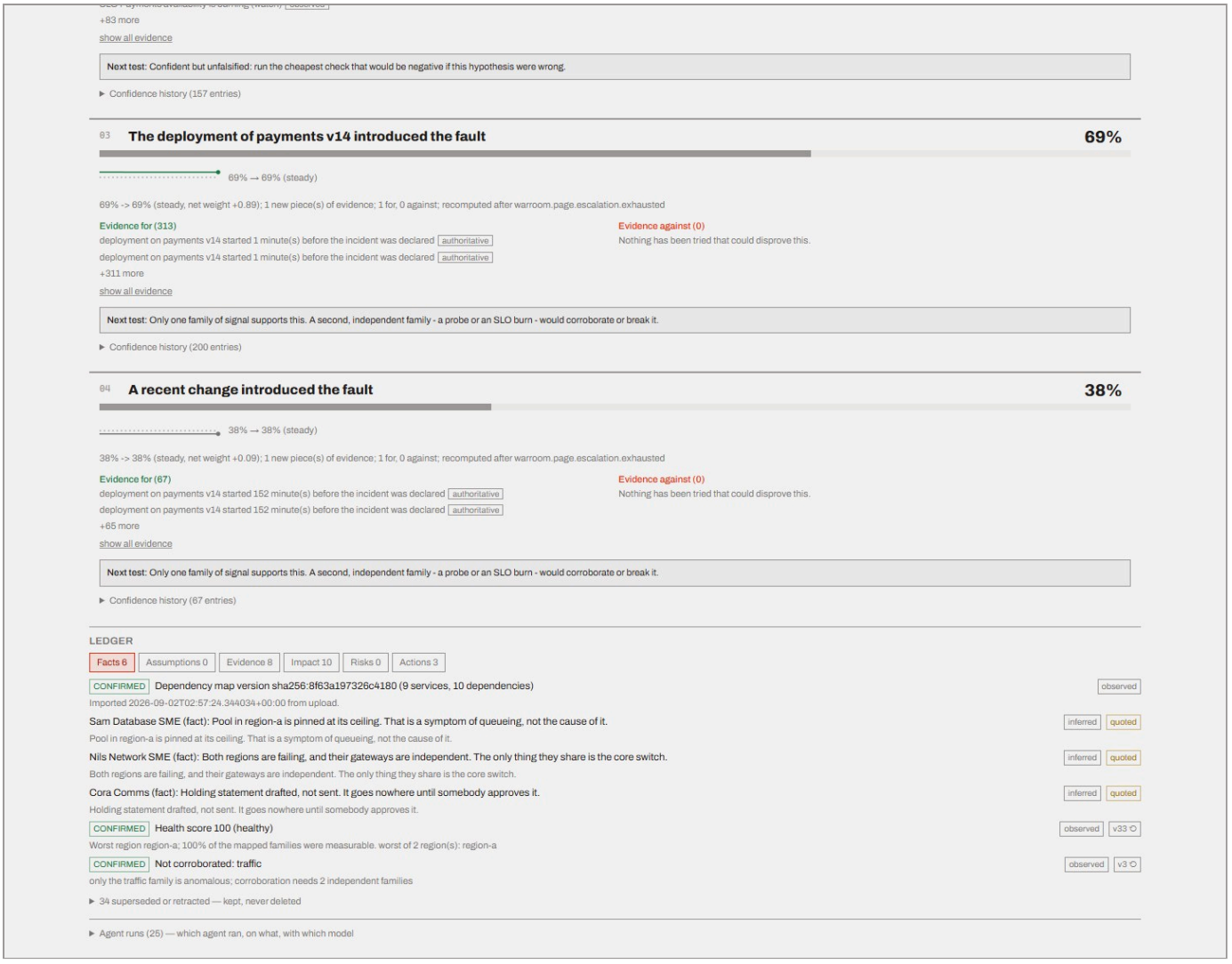
What the platform believes, versioned, with provenance on every node. Each hypothesis carries a confidence computed deterministically, the arithmetic that produced it, the evidence for and the evidence against, and the discriminating test — the one measurement that would move it most. Contradictions between testimony and telemetry are raised, never averaged away.

IN THIS CAPTURE

Four competing hypotheses at 97%, 97%, 69% and 38%, each with its confidence arithmetic expanded, its evidence both ways, and its next test named.



S10 · INCIDENT BRAIN — 1 OF 2



S10 · INCIDENT BRAIN — 2 OF 2

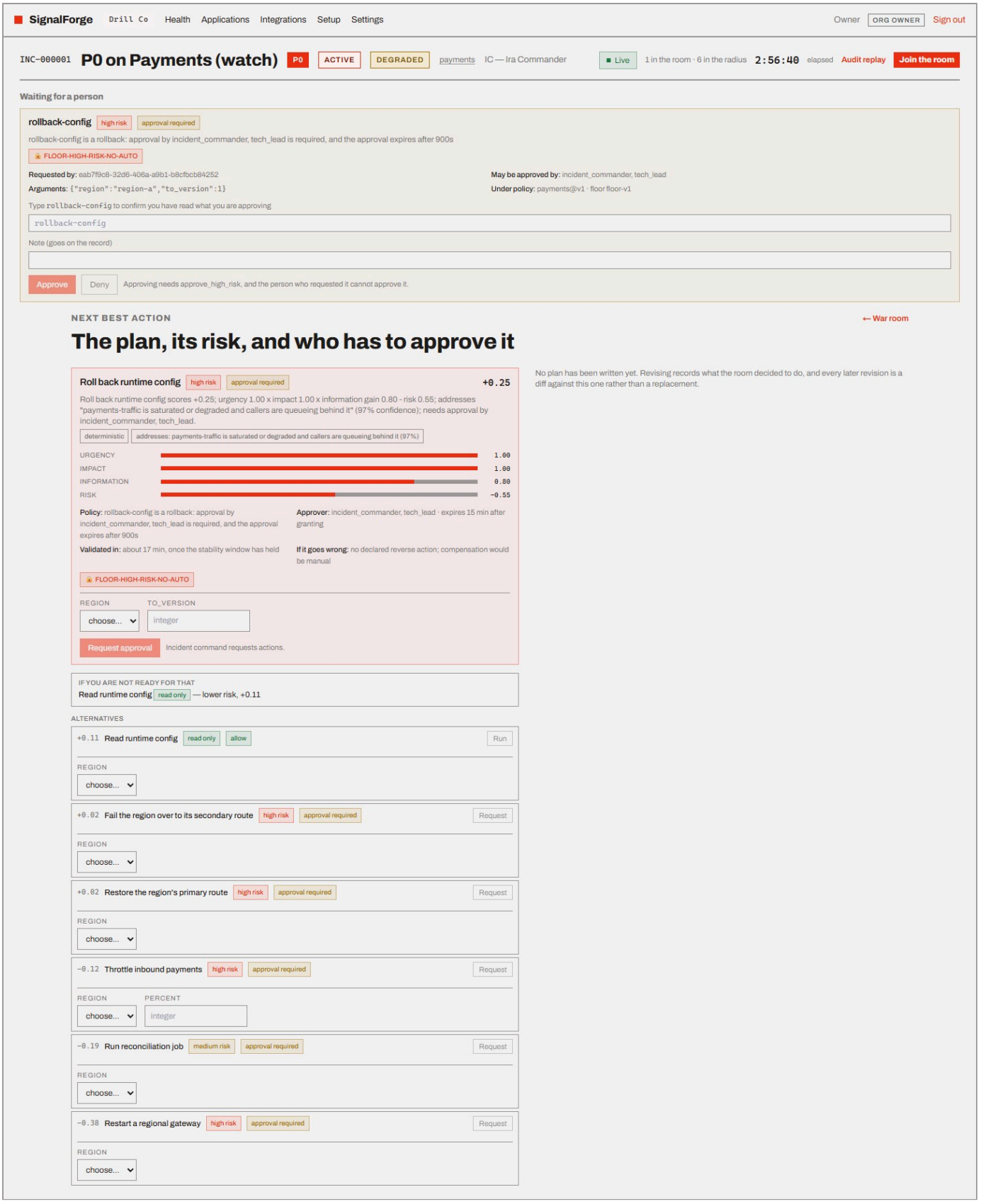
S11 Next best action

`?view=action`

BUILT · VERIFIED LIVE

The ranked plan. Every score is arithmetic and is shown as its terms: urgency × impact × information gain, minus risk. Beside the recommendation sit a lower-risk alternative and a fallback. Arguments are rendered from the application's own parameter schema — an enum becomes a select, everything else a field — and typed on the way out. The platform never invents an argument: a screen that defaulted a region would be choosing which half of an estate to act on.

> **IN THIS CAPTURE**
>
> Roll back runtime config at +0.25 with its four terms drawn out, six alternatives from +0.11 down to −0.38, the region and to_version fields the application's manifest declares, and a FLOOR-HIGH-RISK-NO-AUTO badge. Above it, the approval gate: a typed confirmation, a note that goes on the record, and the statement that the person who requested it cannot approve it.


SignalForge
Drill Co
Health
Applications
Integrations
Setup
Settings
Owner
ORG OWNER
Sign out
INC-000001
P0 on Payments (watch)
P0
ACTIVE
DEGRADED
payments
IC — Ira Commander
Live
1 in the room · 6 in the radius
2:56:40
elapsed
Audit replay
Join the room
Waiting for a person
rollback-config
high risk
approval required
rollback-config is a rollback: approval by incident_commander, tech_lead is required, and the approval expires after 900s
FLOOR-HIGH-RISK-NO-AUTO
Requested by: eab7f9c8-32d6-406a-a9b1-b8cfbcb84252
Arguments: {"region":"region-a","to_version":1}
May be approved by: incident_commander, tech_lead
Under policy: payments@v1 · floor floor-v1
Type rollback-config to confirm you have read what you are approving
rollback-config
Note (goes on the record)
Approve
Deny
Approving needs approve_high_risk, and the person who requested it cannot approve it.
NEXT BEST ACTION
← War room
The plan, its risk, and who has to approve it
Roll back runtime config
high risk
approval required
+0.25
Roll back runtime config scores +0.25; urgency 1.00 x impact 1.00 x information gain 0.80 - risk 0.55; addresses "payments-traffic is saturated or degraded and callers are queueing behind it" (97% confidence); needs approval by incident_commander, tech_lead.
deterministic
addresses: payments-traffic is saturated or degraded and callers are queueing behind it (97%)
URGENCY 1.00
IMPACT 1.00
INFORMATION 0.80
RISK -0.55
Policy: rollback-config is a rollback: approval by incident_commander, tech_lead is required, and the approval expires after 900s
Approver: incident_commander, tech_lead · expires 15 min after granting
Validated in: about 17 min, once the stability window has held
If it goes wrong: no declared reverse action; compensation would be manual
FLOOR-HIGH-RISK-NO-AUTO
REGION
TO_VERSION
choose...
integer
Request approval
Incident command requests actions.
No plan has been written yet. Revising records what the room decided to do, and every later revision is a diff against this one rather than a replacement.
IF YOU ARE NOT READY FOR THAT
Read runtime config
read only
— lower risk, +0.11
ALTERNATIVES
+0.11 Read runtime config
read only
allow
Run
REGION
choose...
+0.02 Fail the region over to its secondary route
high risk
approval required
Request
REGION
choose...
+0.02 Restore the region's primary route
high risk
approval required
Request
REGION
choose...
−0.12 Throttle inbound payments
high risk
approval required
Request
REGION
PERCENT
choose...
integer
−0.19 Run reconciliation job
medium risk
approval required
Request
REGION
choose...
−0.38 Restart a regional gateway
high risk
approval required
Request
REGION
choose...

S12 Execution ledger

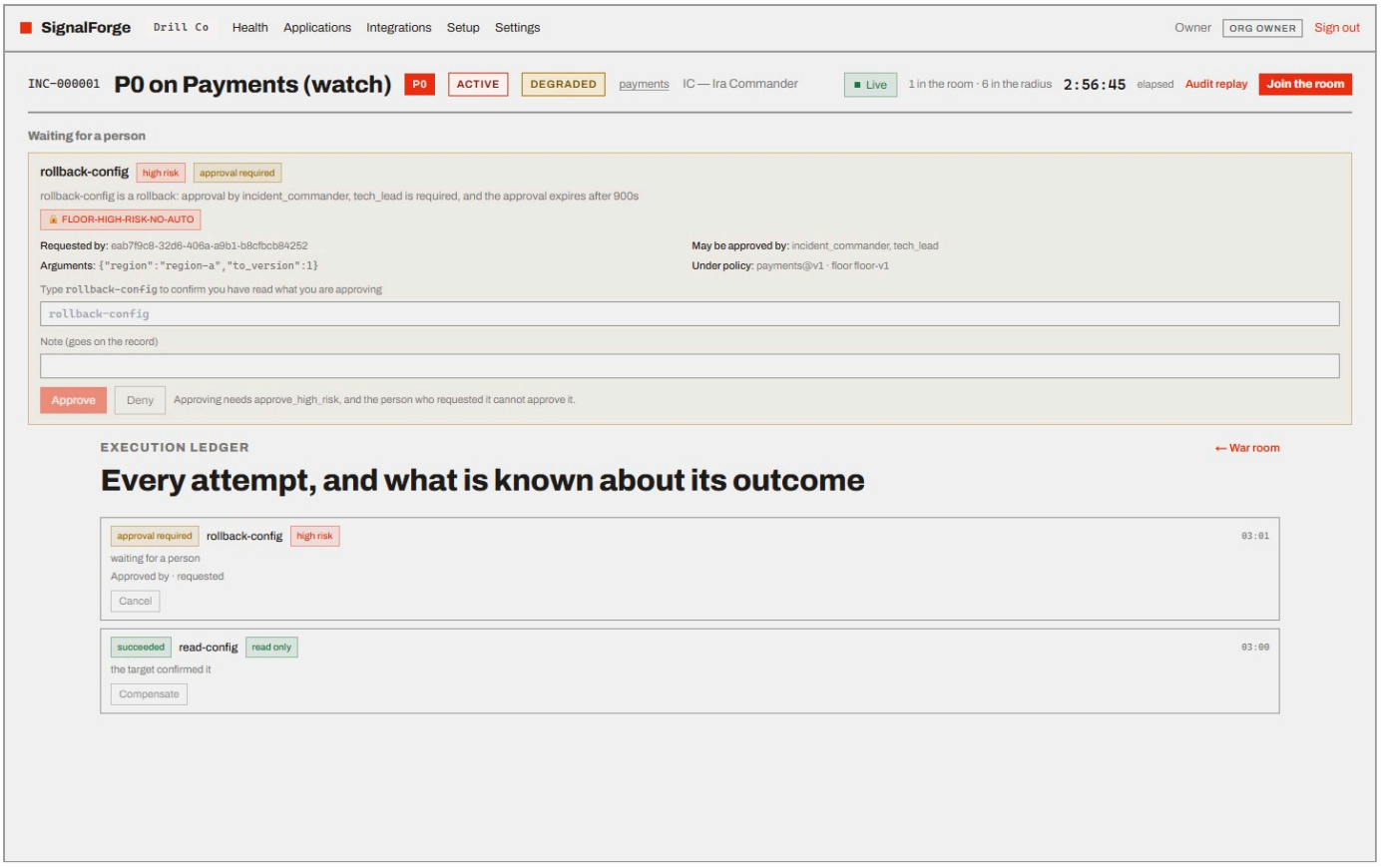
BUILT · VERIFIED LIVE

`?view=ledger`

Every action ever attempted on this incident, its state, who requested and who approved it, and what came back. UNCERTAIN is the state that matters: it does not mean the action failed, it means nobody knows. Retry is refused until the platform reconciles — it asks the application what actually happened, and the answer is COMPLETED_LATE, NOT_APPLIED or PARTIAL.

IN THIS CAPTURE

A rollback awaiting approval and a completed read-only action.



S13 Blast radius

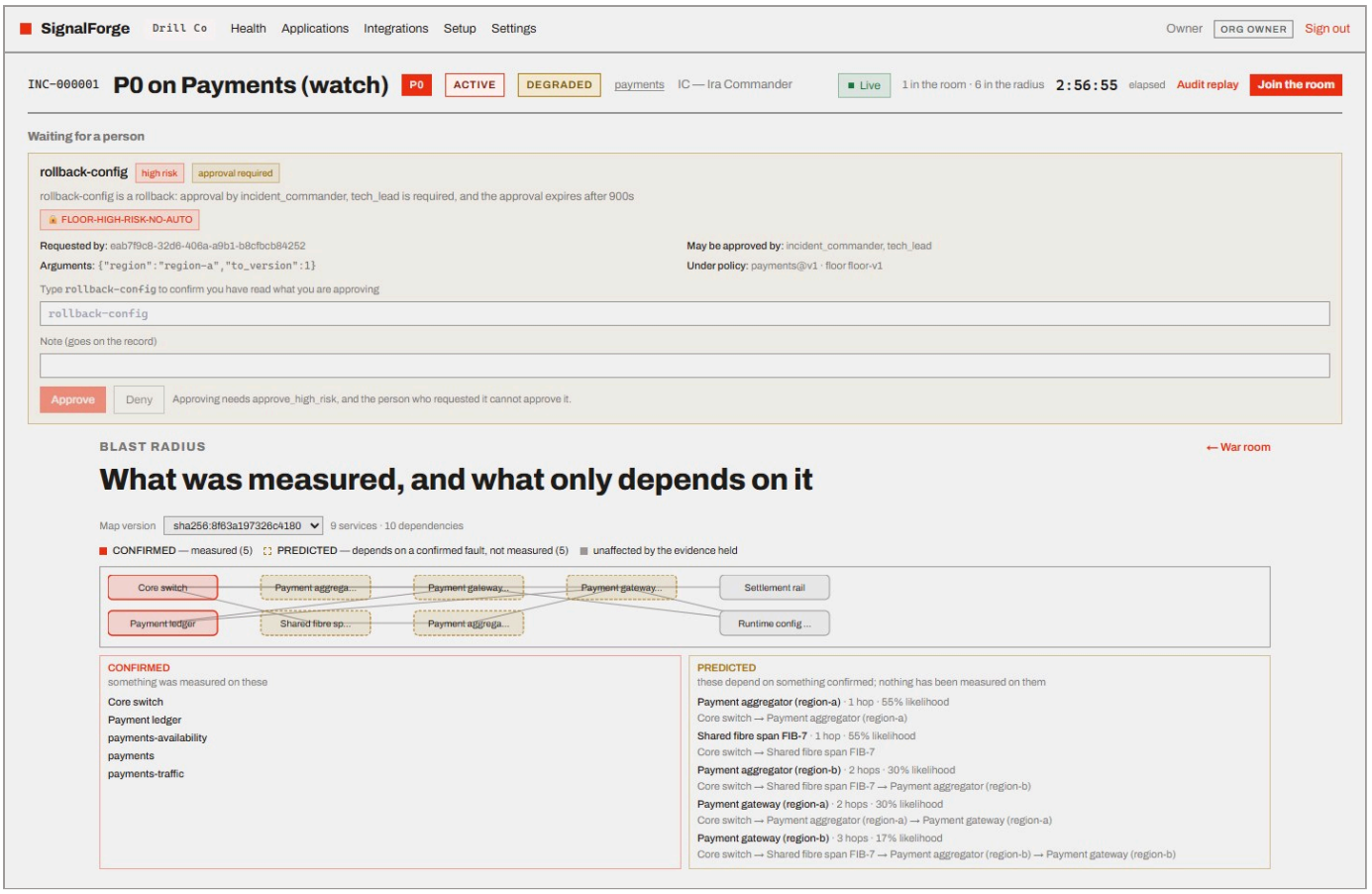
`?view=radius`

BUILT · VERIFIED LIVE

Two lists that are never merged. CONFIRMED is what was measured. PREDICTED is what depends on something confirmed and has not been measured, with the hop count and the path that produced it. The map is addressed by content version.

IN THIS CAPTURE

Nine services and ten dependencies at map version sha256:8f63a19..., five CONFIRMED and five PREDICTED at one to three hops with likelihoods from 55% down to 17%, each naming its path.



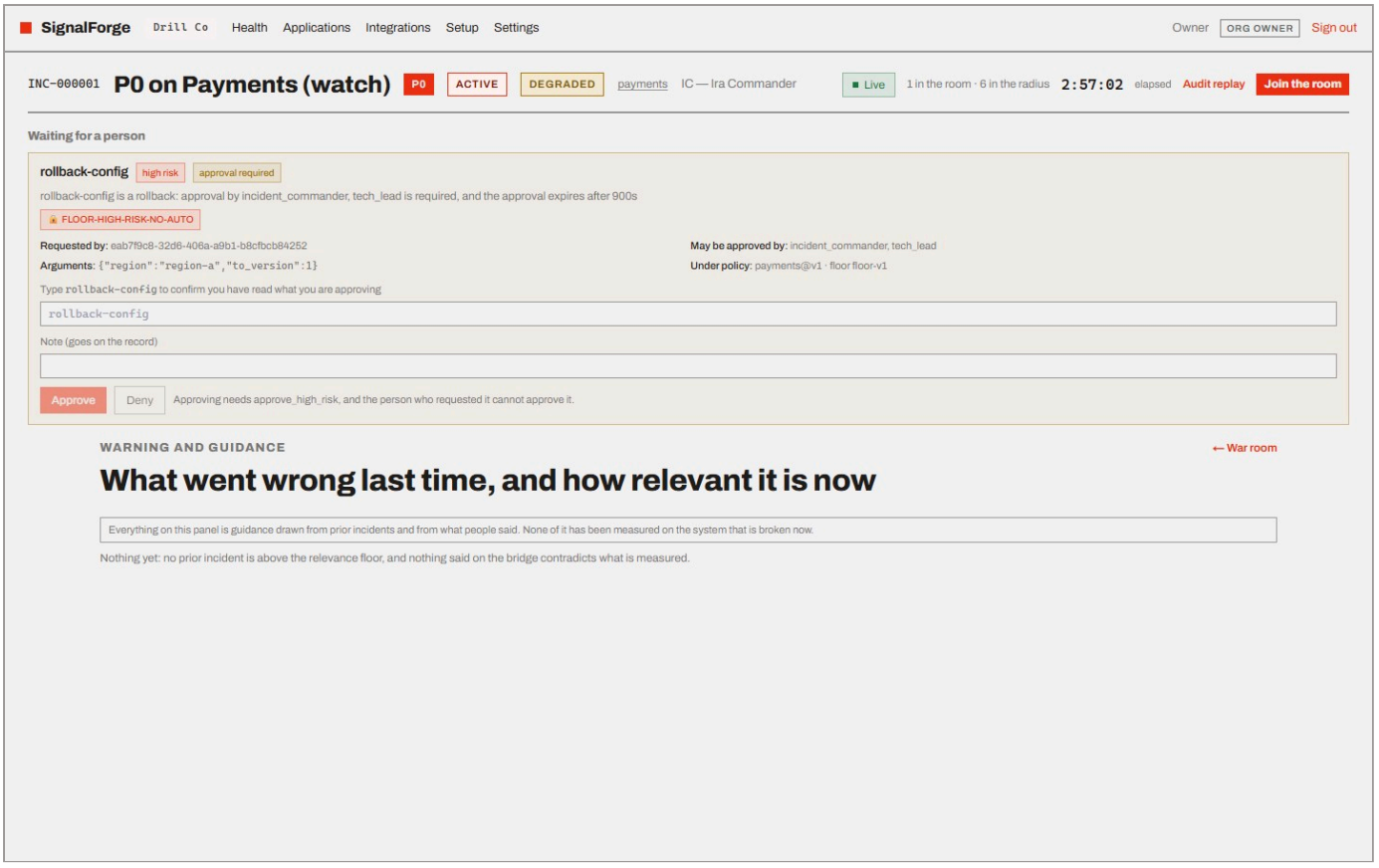
S14 Warning and guidance

`?view=guidance`

BUILT · VERIFIED LIVE

What the room should be careful about, drawn from what this incident has produced and from what previous incidents recorded. Guidance cites; it does not assert.

> **IN THIS CAPTURE**
> The panel with its history-and-relevance link. Nothing yet meets the bar to warn about.



S15 Communications

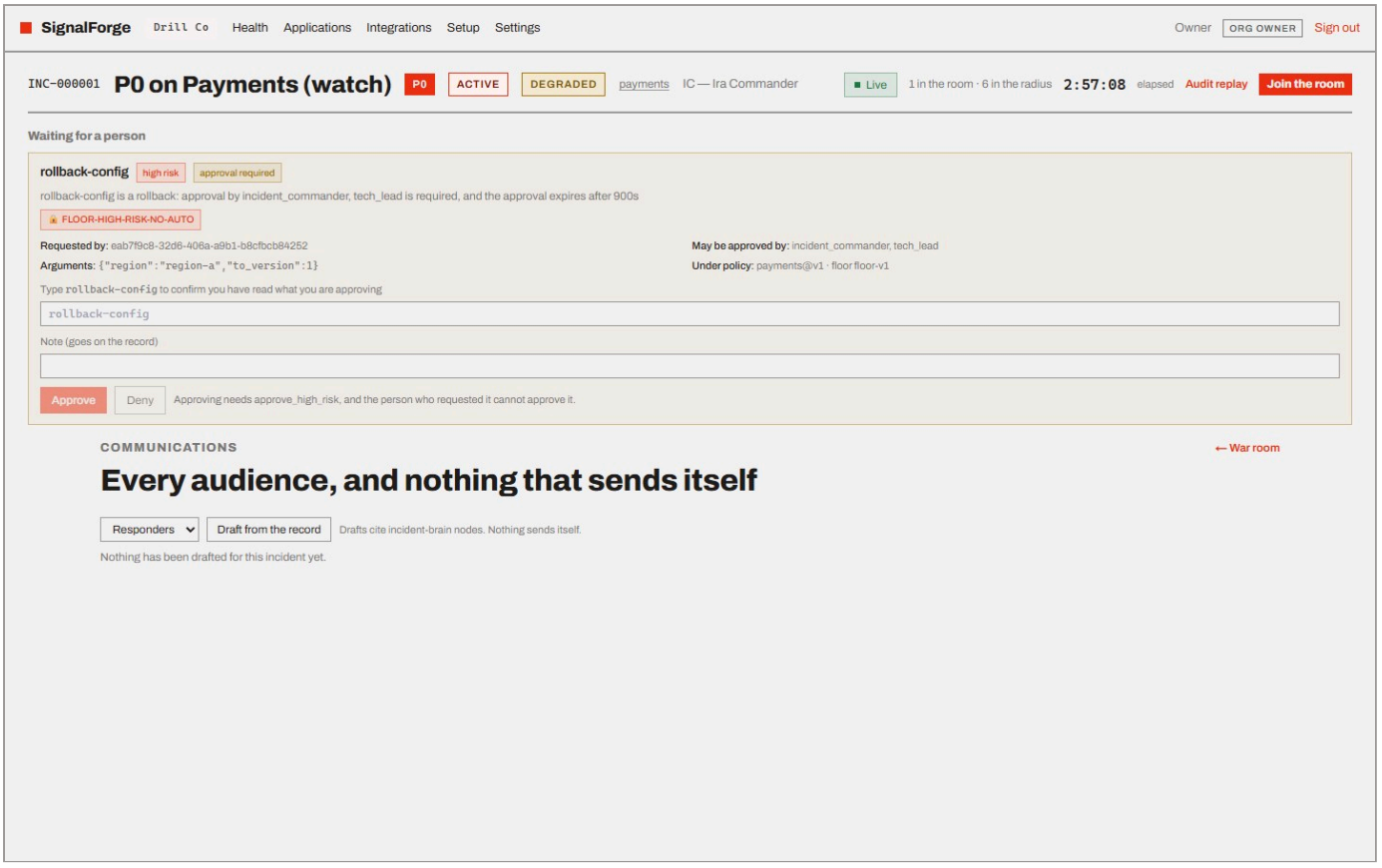
`?view=comms`

BUILT · EMPTY IN THIS CAPTURE

Nothing sends itself. A communication is drafted — grounded on graph nodes it cites — approved by somebody other than its author for external audiences, and then issued by a person. If the evidence a published statement rests on is later superseded, the statement is flagged contradicted and its text is left exactly as published; correcting it is a new communication with a redline and its own approval.

IN THIS CAPTURE

Audiences and drafting controls, with nothing drafted yet on this incident.



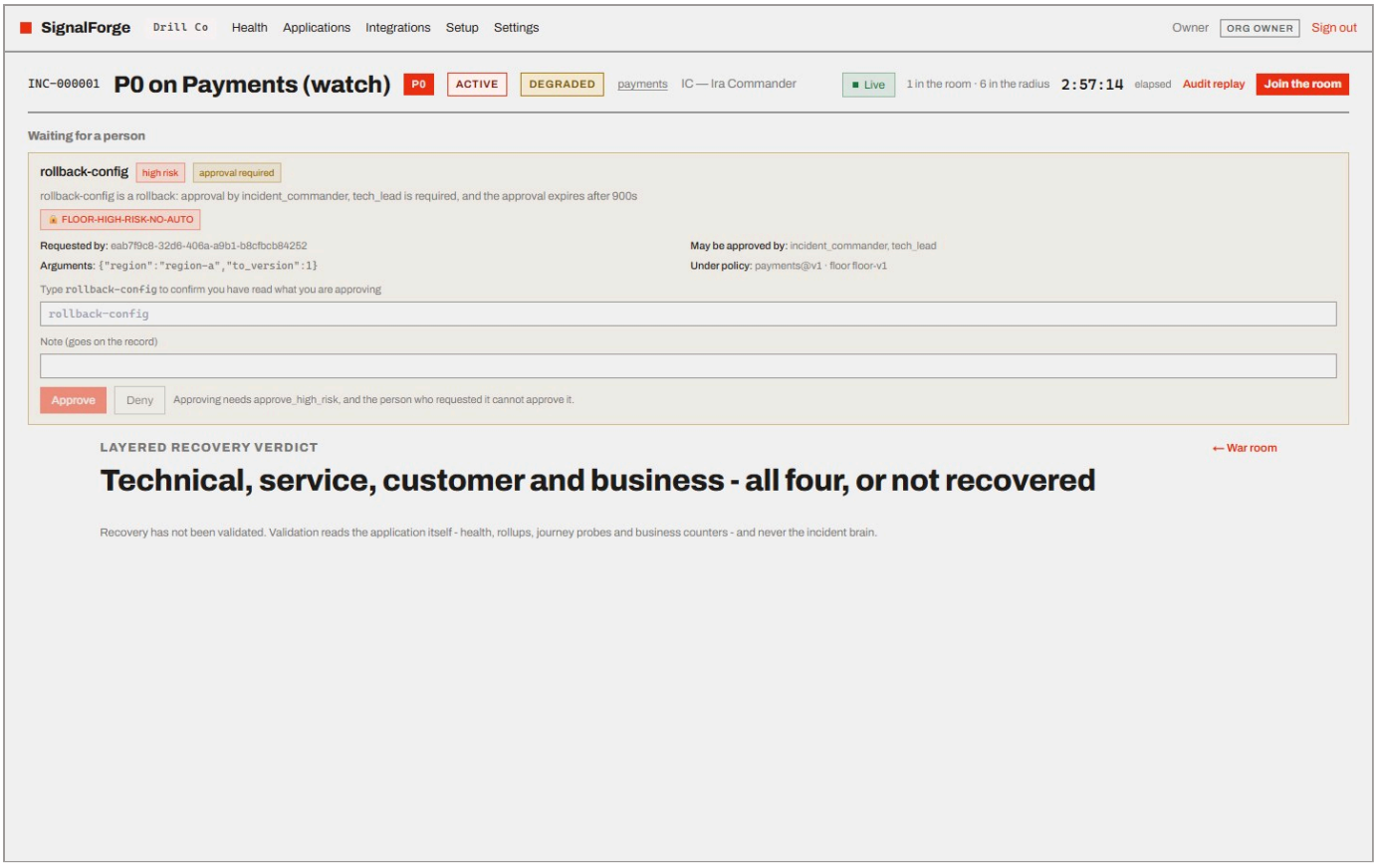
S16 Recovery verdict

`?view=verdict`

BUILT · EMPTY IN THIS CAPTURE

Four independent layers — technical, service, customer and business — that read the application directly and never ask the brain. The customer layer blocks closure: technical green while customers still cannot pay is not recovery, and the platform refuses the close with a reason.

> **IN THIS CAPTURE**
> Not yet validated on this incident; the screen says so and names what validation reads.



S18 The IC copilot

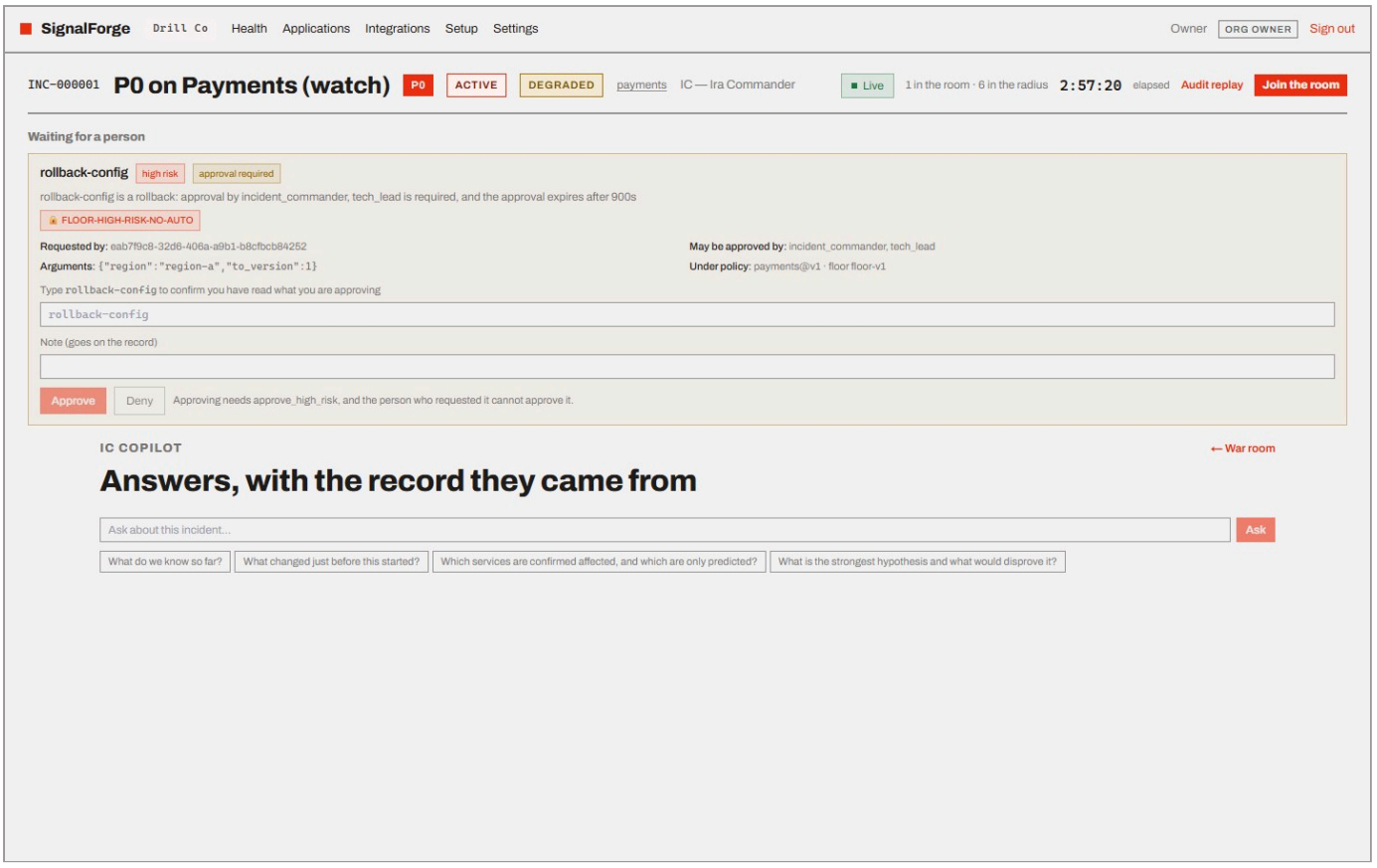
BUILT · VERIFIED LIVE

`?view=copilot`

Grounded question answering over the incident graph. It answers with citations or it says it cannot answer; untrusted text stays untrusted and is never followed as an instruction. With no model configured it degrades to deterministic text and says so.

IN THIS CAPTURE

The suggested-question surface, with the model path in DEGRADED state on this instance.

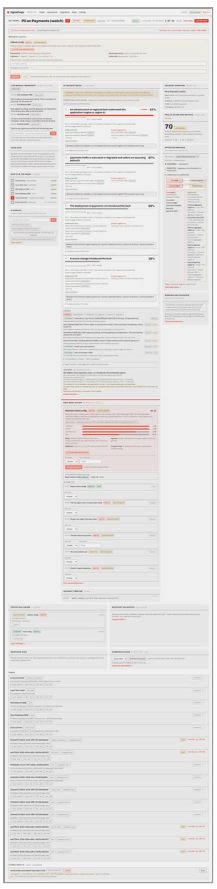


SECTION E

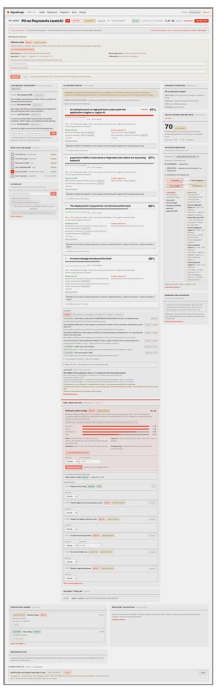
The same room, per role

> Presence, not position. A panel that stays is where the full cockpit puts it, so somebody who learns where the bridge is as an SME finds it there the day they are made commander. readOnly removes controls and keeps information — an executive pressing "run this action" during a P0 is a failure mode, but a view that hid the evidence in order to justify hiding the button would be a different product. This is presentation, not authorisation: the API decides what a role may see, and it is unchanged.

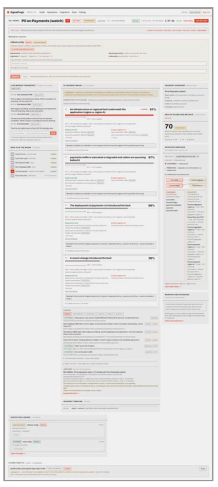
Incident Commander
`6,082px`



Tech Lead
`4,702px`



SME
`3,288px`



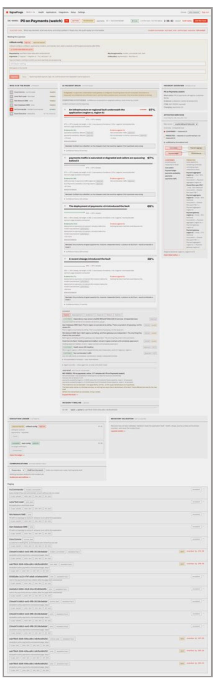
Comms Lead
`3,391px`



Executive
`2,280px`



Auditor
`4,640px`



ROLE	WHAT THE ROOM BECOMES	WHAT THE ROLE MAY DO
Incident Commander	Everything, with every control.	`declare · change severity · approve high-risk · close`
Tech Lead	Everything operational; no closure.	`declare · change severity · approve high-risk`
SME	The evidence and the bridge. Deliberately holds no permissions — answering a page is self-scoped.	`answer pages · investigate`
Comms Lead	The room's state and the communications board.	`approve customer communications`
Executive	The shortest view: impact, state, communications. No operational control.	`approve executive communications`
Auditor	The record, and nothing else, ever.	`read the audit`

Six views of the same incident at the same moment, all drawn to the same width, so the height of each column is what the role is actually given. The Incident Commander gets 6,082 pixels of room; the Executive gets 2,280 of the same room, with every operational control removed and none of the evidence hidden. Nothing here is a mock-up of a permission: each is `/incidents/{id}/as/{role}` as it rendered.

SECTION F

Afterwards

> The record is a hash chain. Everything below depends on it verifying, so the screen says whether it does before it says anything else.

S20 Audit replay and review

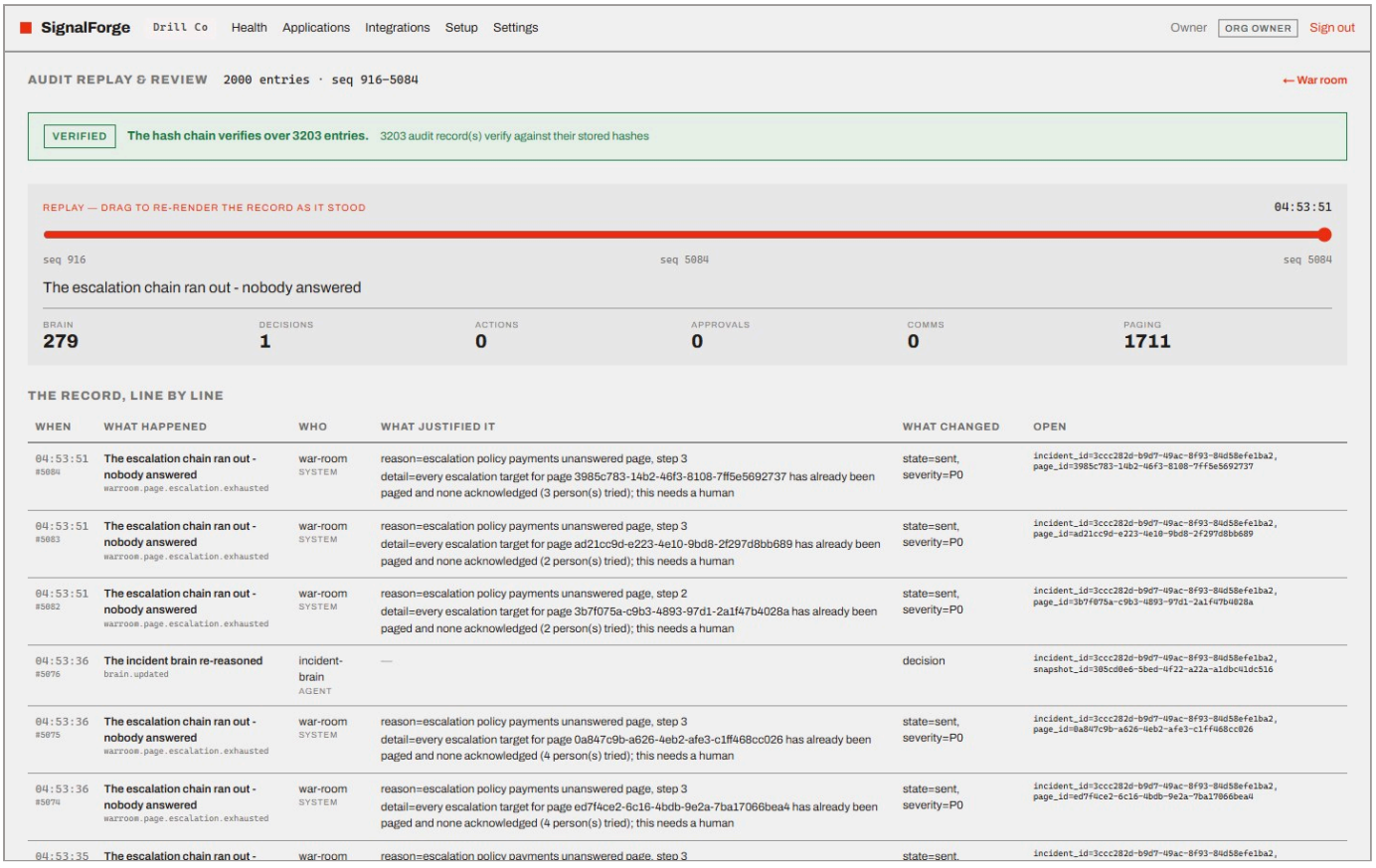
`/incidents/{id}/replay`

BUILT · VERIFIED LIVE

The integrity banner first and never a footnote. The seq scrubber is a server parameter (?until_seq=), so a scrubbed view is a byte-prefix of the record rather than a subset the screen chose — a partial view cannot disagree with the whole. Six columns: when, what happened, who or what caused it, what justified it, what changed, and references you can open. An em dash where the record has nothing, because "we did not record that" and "nothing changed" are different findings. The post-incident review generates sixteen sections from the record and refuses to generate at all if the chain does not verify; its lessons are VERIFIED only when every citation resolves, and a model may suggest a lesson but can never promote one.

IN THIS CAPTURE

VERIFIED — the hash chain verifies over 3,203 entries. 2,000 entries in view, seq 916–5084, with the scrubber at the head; 279 brain events, 1 decision, 1,711 paging events.



SECTION G

Operating the platform

> Day-to-day surfaces for the people who run SignalForge itself rather than an incident.

S21 Settings

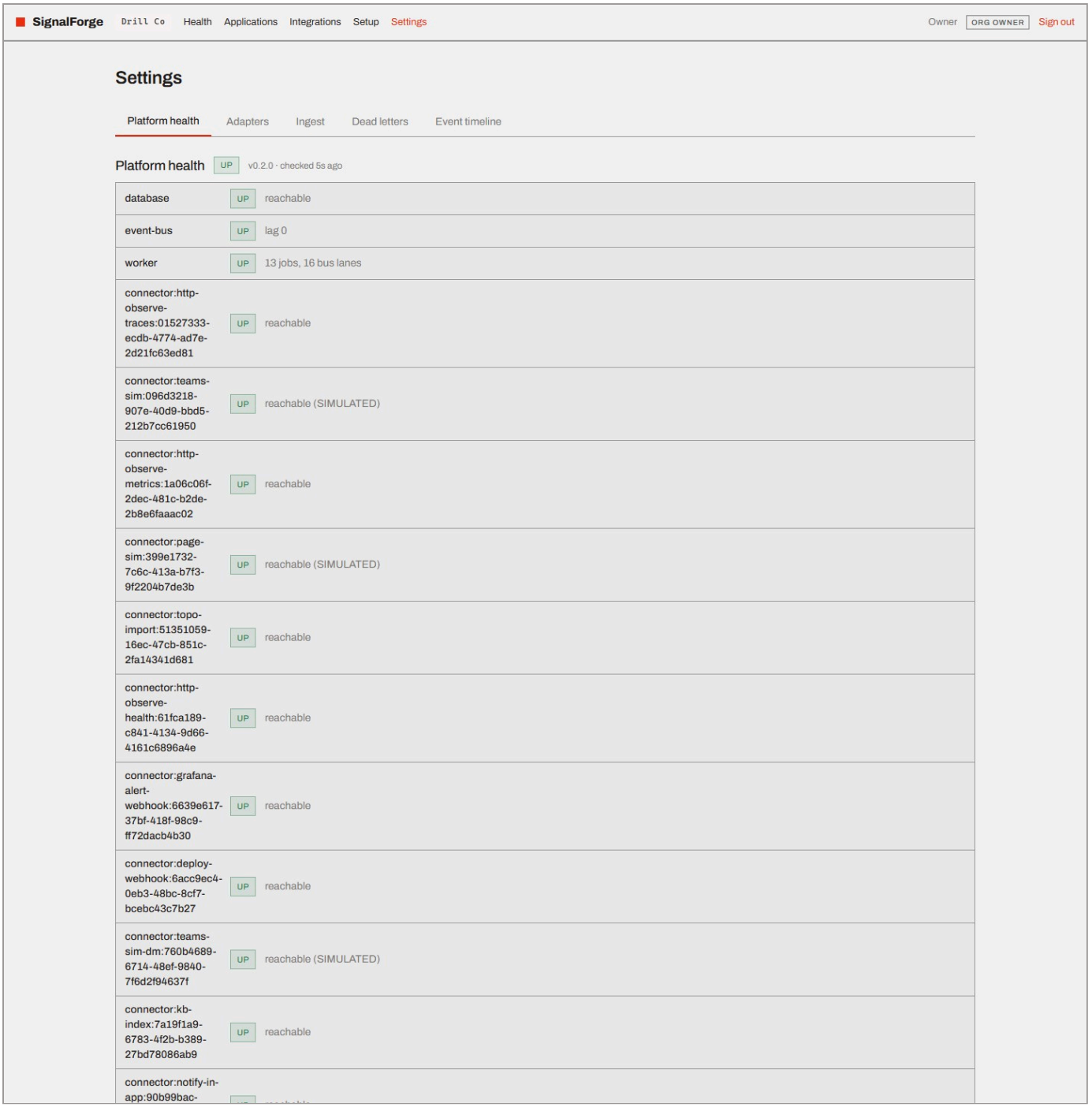
`/settings`

BUILT · VERIFIED LIVE

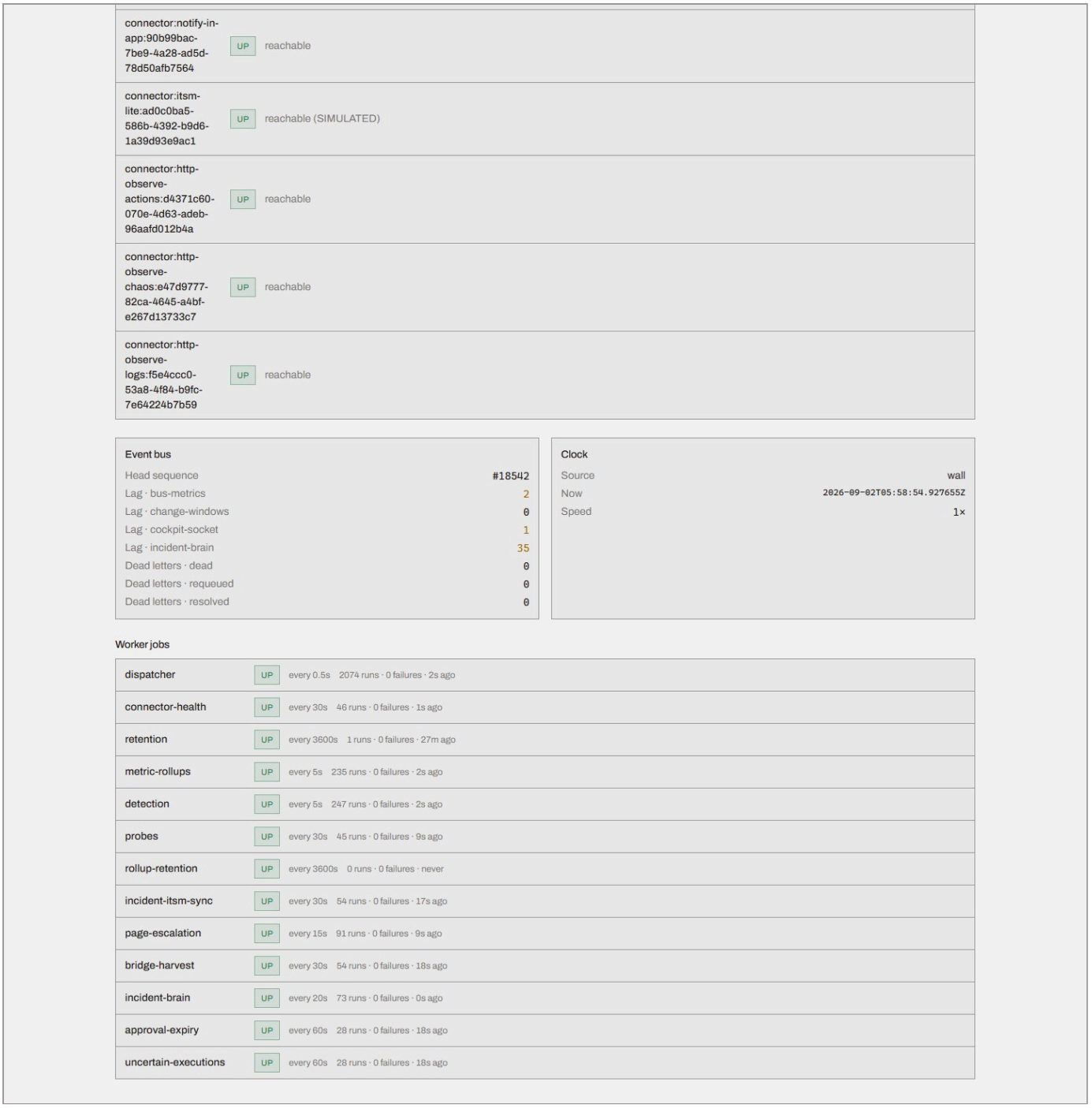
Five tabs: platform health with per-source detail, the adapter registry and its capability matrix, ingest keys with rotation, dead letters with requeue, and the raw event timeline.

IN THIS CAPTURE

Database, event bus and worker all UP — 13 jobs across 16 bus lanes — and every bound connector reachable, with the simulated ones labelled.



S21 · SETTINGS — 1 OF 2



S21 · SETTINGS — 2 OF 2

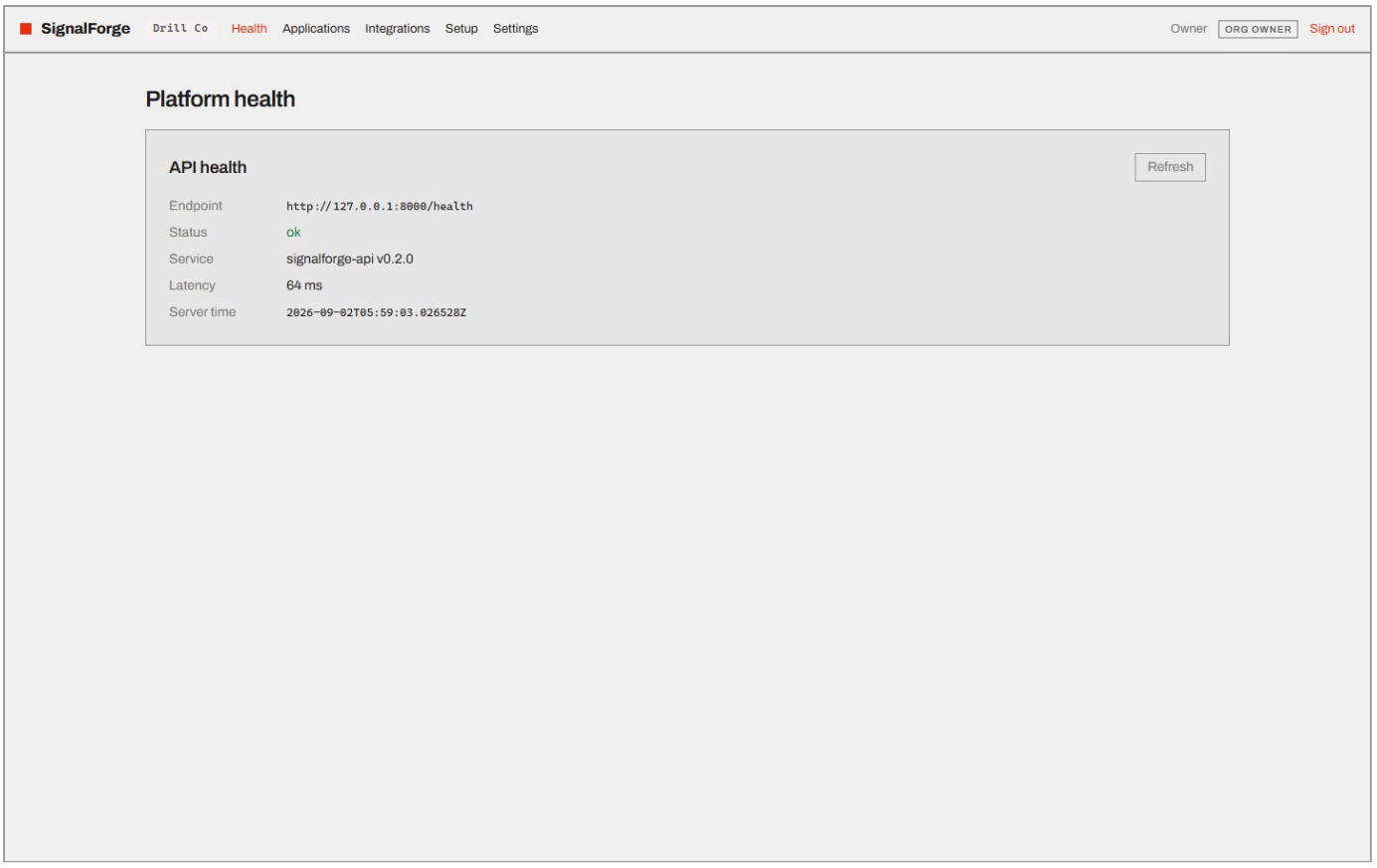
S0 Platform health

`/health`

BUILT · VERIFIED LIVE

Liveness with per-source detail, the same shape the API serves at /health. /metrics carries the Prometheus series; the one worth an alert is signalforge_policy_floor_rejections_total, because non-zero means a control actively prevented an unsafe configuration.

> **IN THIS CAPTURE**
> All components up.



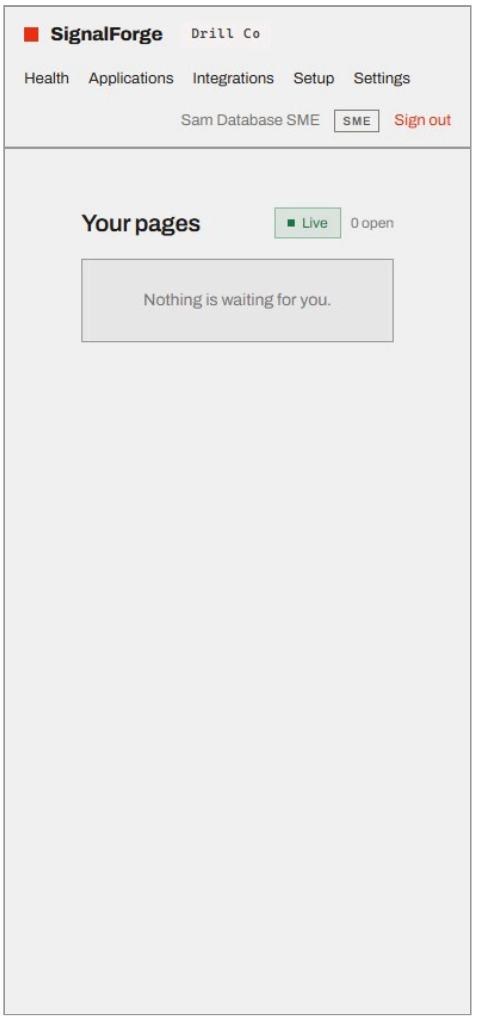
S22 Mobile paging

`/my/pages @ 390px`

BUILT · EMPTY IN THIS CAPTURE

The pager, at phone width. Acknowledging is self-scoped — the page names its recipient and the session names itself — so it is refused for somebody else's page whatever roles you hold. An unanswered page escalates to the next person in the rotation on its own.

> **IN THIS CAPTURE**
> Signed in as Sam Database SME: nothing waiting. Every page on this incident had already escalated through its policy hours before the capture, which is the escalation working rather than the screen being empty.



S22b The war room at 390px

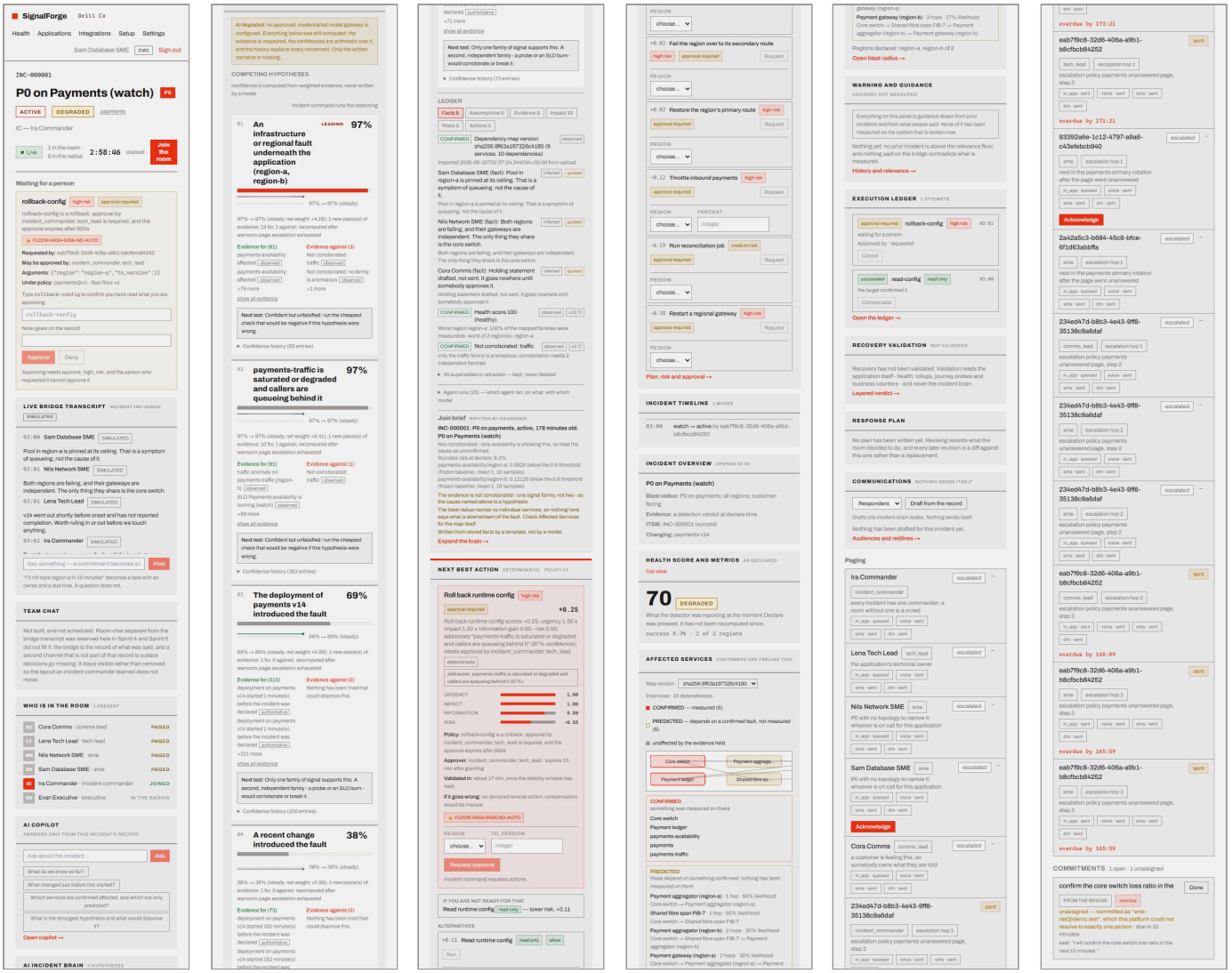
`/incidents/{id} @ 390px`

BUILT · VERIFIED LIVE

The cockpit is responsive: the three-column grid stacks into one column and every panel survives. Shown here as six consecutive panels of one 12,264px capture.

IN THIS CAPTURE

The whole room, in order, on a phone.



SECTION H

What is not built

The honest list. Each entry says what is absent and why, because a deferral with a reason is a decision and a deferral without one is a hole.

DELIBERATELY NOT A SCREEN

S19 — a drill console

The drill engine is structurally unreachable from the API: no published route names a drill, chaos or an injection, and the API package does not import the drill engine at all. Both are asserted by tests. The console is scripts/demo.sh and scripts/drill_run.py. A drill route would trade a structural guarantee for a page.

DEFERRED, WITH A REASON

A sparkline on the golden tiles

The detection route serves a reading per region with its baseline and threshold — not a history — so the only line a tile could draw is one point per region, which renders as a trend and is not one. A chart reading "this metric is rising" when it means "region-b is higher than region-a" is worse than no chart. /apps/{id}/rollups already serves the history; the tile earns a sparkline when it reads that.

NOT BUILT

A Drill Author agent

Drills are written as data, by hand. There is no agent that authors them.

NOT BUILT, ON PURPOSE

A second topology version in the demonstration world

The reference application derives its edges from its live routing table, so its map changes when a route changes — and no fault changes a route. What does is the failover-route action, which is high risk and therefore something a person does during the incident. Uploading a hand-edited document would have produced a prettier Affected Services delta and a false one.

READY, UNTESTED

Postgres

Single node, SQLite in WAL mode, through SQLAlchemy 2 async and Alembic. The schema and the session layer are Postgres-ready. No Postgres run has been made, so no claim is.

PARTIALLY VERIFIED

The release archive, booted end to end

dist/signalforge-6e6f2f1.zip was unzipped into an empty directory, its contents checked against the exclusion list (no .env, var/, node_modules, .next, .venv, .git, target/), its entry points confirmed, and its reset.sh run successfully from inside it. A full run.sh from the archive needs npm ci and a Maven build of the harness and was not done.

BLOCKED / DEFERRED

Dependency audits

next and postcss carry high advisories whose remediation is a major Next upgrade, deferred deliberately. pip-audit could not run on the build machine because it intercepts TLS, so no claim is made about the Python dependency set.

MEASURED, BUT NOT A PRODUCTION CLAIM

Performance at scale

The figures in docs/17 are single-process and in-process. They are not a production-scale claim and that document says so.

SECTION I

What this walkthrough found

Five observations from driving every route. Two are documentation defects worth fixing; three are context a reader of the screenshots needs in order to read them correctly. None is a defect in product code.

WORTH FIXING

The shipped wizard screenshots are both the same step

docs/screens/S03-wizard-connect.png and S03-wizard-map.png are byte-identical in size (76,740 bytes) because the capture script addressed the wizard by step name (/onboard/connect, /onboard/map) and the route takes a number — both requests fell back to step 1, Basics. The seven step captures in section B of this document were taken with the numeric routes and are distinct. Product code is unaffected; the fix is one line in playbook/design/capture-product.cjs.

WORTH FIXING

The committed RTM was generated from a partial test run

docs/20-RTM.md reports "Requirements: 52 | with linked tests: 1 | without: 51 | Tests recorded: 8". That is not the state of the suite — it is the state of the last run make rtm read, which was a run of the detection tests alone. Counted directly from the source, all 52 requirements carry @pytest.mark.req markers across 908 marker occurrences in 170 test files. Regenerating the RTM after a full run would correct the document; the traceability itself is intact.

CONTEXT

The AI surfaces were DEGRADED on the captured instance

Every war-room capture wears a DEGRADED badge, which is the product behaving as designed: with no model reachable, narratives fall back to deterministic text and system.llm.degraded appears on the timeline. Detection, hypothesis ranking, policy, the Tool Gateway, validation and replay never used the model in the first place, and every one of them is populated in these captures. The badge records the state of this machine's gateway configuration at capture time, not a missing feature.

CONTEXT

Three panels were photographed empty, and they are empty for a reason

Communications has nothing drafted, recovery validation has not been run, and the mobile pager has nothing waiting because every page on this incident had already escalated through its policy. Each is a reachable state of a working screen rather than an unbuilt one; each is labelled EMPTY in this document rather than shown as built with data it does not have.

CONTEXT

The demonstration world heals itself

Every chaos scenario carries a max_duration_s the reference application enforces, so a walkthrough running longer than ten minutes finds a healthy application. That is why five of six golden tiles read healthy in these captures beside a live P0 declared three hours earlier. Re-arm with python scripts/drill_run.py drills/demo-setup.yaml --refault; it does not touch the incident record.

APPENDIX

The surface underneath the screens

> Every screen in this document is a client of the same public API an onboarded application talks to. There is no private route behind the cockpit.

API, 107 paths

GROUP	PATHS	WHAT IT CARRIES
`/auth, /orgs, /users, /invites`	11	Sign-up, sessions, the role matrix, invitations, WebSocket tickets
`/apps`	20	Onboarding, mapping, discovery, connection tests, policy, readiness, detection, rollups
`/adapters, /connectors`	6	The registry, the capability matrix, bindings and connector health
`/ingest-keys, /api/v1/ingest`	8	HMAC keys with rotation, and the five push endpoints an app posts to
`/incidents`	42	The whole incident: brain, bridge, pages, executions, comms, tasks, validation, replay, PIR
`/oncall, /escalation, /my`	6	Rotations, escalation policies and a person's own pages
`/events, /platform`	10	Event stream, seals and chain verification; agent runs, dead letters, status
`/health, /metrics`	2	Liveness with per-source detail, and the Prometheus series

Code, by layer

LAYER	RULE IT OBEYS	FILES	LINES
contracts	Frozen. Referenced, never redefined.	9	1,213
domain	Entities, value objects, ports. No framework import.	113	20,473
application	Use cases. Orchestrates domain through ports.	83	22,575
infrastructure	Implements domain ports. All vendor code lives here.	87	15,504
api	Thin routers, pydantic schemas, /ws.	41	9,306
workers	Scheduled jobs on the same use cases.	7	476
web	Route files thin; logic in features/. One WebSocket store.	167	21,968
tests	REQ-marked. The RTM is generated from them.	170	35,091

Dependencies point inward: api → application → domain, with infrastructure implementing domain ports. Domain code imports no FastAPI, no SQLAlchemy, no Next.js and no vendor SDK. `scripts/check_clean.sh` additionally fails the build if any application-specific string appears outside the reference harness, the drills, the seeds and the tests — the guarantee that SignalForge is not built around the one application it was demonstrated on.

How to reproduce every picture in this document

```
bash reset.sh && bash run.sh
bash scripts/demo.sh
python scripts/demo_check.py
node playbook/design/capture-product.cjs <org-slug> <incident-id>
```

The demonstration world is data — `drills/demo-setup.yaml` — and deleting it leaves a working product. The reference harness under `reference-app/` is one example of an onboardable application and is never imported by product code.